Test 22/05 Run Report

Test 22/05 Run

RUN TIMESTAMP	JUDGE
2026-04-22T08:03:04.367093+00:00	Claude Opus 4.7

PROMPTS	CANDIDATES
9	6

⚠ **Potential judge bias:** The judge (**Claude Opus 4.7**) appears to share a model family with candidate(s) **Claude Opus 4.7**. LLMs are known to exhibit self-preference bias. Consider re-running with a different-family judge for an independent baseline.

EXECUTIVE SUMMARY

Global Score formula: 70% Judge Score (holistic quality) +

15% Cost Score (normalised inverse) +

15% Performance Score (latency + throughput, normalised)

CANDIDATE	JUDGE SCORE	QUALITY SCORE	TOTAL COST	AVG LATENCY	GLOBAL SCORE	Δ VS LEADER
★ **gpt-5.4-mini**	90.06	93.96	$0.02	**2713 ms**	**92.65**	—
gpt-5.4	**92.50**	**95.97**	$0.06	5446 ms	**84.21**	-8.44
Claude Sonnet 4.6	91.61	94.36	**$0.00**	11907 ms	**81.98**	-10.67
OVH gpt-oss-120b	88.11	92.70	$0.01	17888 ms	**80.96**	-11.69
gpt-5.4-nano	75.00	83.93	$0.01	5103 ms	**77.37**	-15.28

CANDIDATE	JUDGE SCORE	QUALITY SCORE	TOTAL COST	AVG LATENCY	GLOBAL SCORE	Δ VS LEADER
Claude Opus 4.7	90.67	93.22	$0.58	9446 ms	**69.95**	-22.70

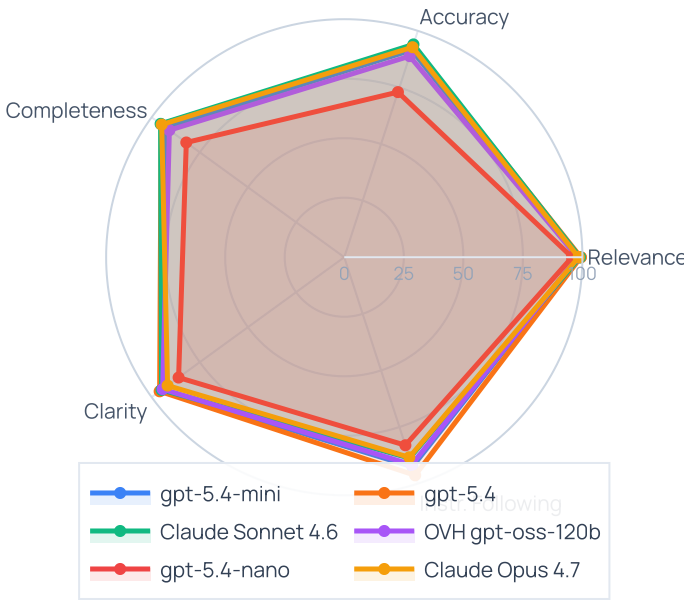
SCORE BY CATEGORY

CATEGORY	GPT-5.4-MINI	GPT-5.4	CLAUDE SONNET 4.6	OVH GPT-OSS-120B	GPT-5.4-NANO	CLAUDE OPUS 4.7
Coding	94.0	**94.3**	93.0	89.3	60.0	91.7
Reasoning	75.5	80.0	85.0	85.0	54.5	**86.0**
Structured Output	**99.5**	98.5	96.0	97.0	93.0	96.0
Summarization	**100.0**	99.0	95.0	99.0	99.0	99.0
Writing	62.0	82.0	**90.0**	55.0	62.0	55.0

VISUAL ANALYSIS

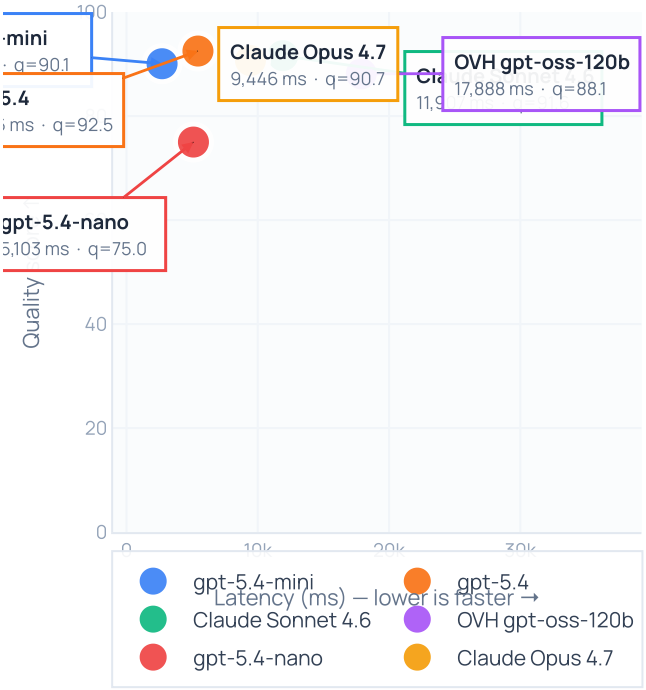
Sub-Score Radar

Dimension-level averages — shows the "personality" of each model.



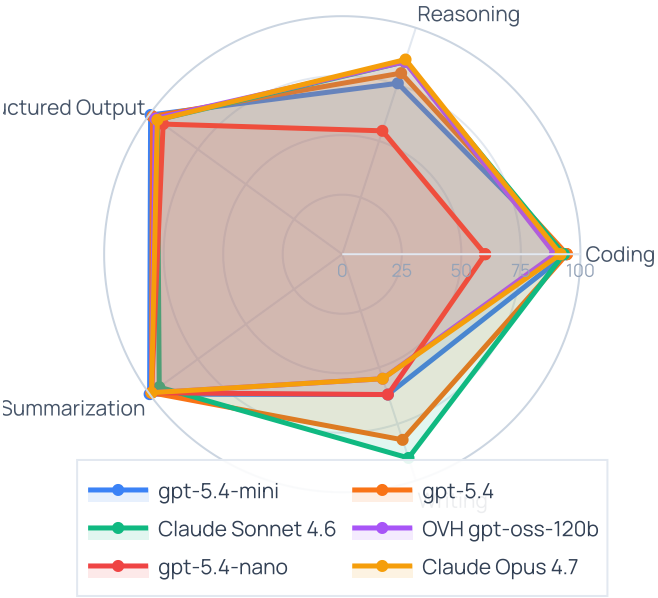
Efficiency Frontier — Latency vs Quality

Upper-left is ideal: high quality at low latency.



Category Efficiency Radar

Average judge score per prompt category.



CANDIDATE SECTIONS

gpt-5.4-mini

openai / remote

92.65	2713 ms	112.91 tok/s	$0.02
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	98.00
Accuracy	91.56
Completeness	92.67
Clarity	95.00
Instruction Following	92.56

Average overall score 90.06 across 9 prompts with mean latency 2713 ms.

STRENGTHS

Correct isolation: writes stored per-transaction until commit; Snapshot uses dict() copy ensuring immutability; Distinct monotonic counters for txn and snapshot IDs

WEAKNESSES

read_committed and read_snapshot raise KeyError on non-open txn is not applicable, but commit/rollback on already-finalized txn raise KeyError which matches spec; The 3-sentence explanation for last-writer-wins is slightly superficial—doesn't emphasize that commit order determines winner via timestamp-free overwrite; Minor: re-entrant calls not supported since using non-RLock, but spec says only Lock so fine

gpt-5.4

openai / remote

84.21	5446 ms	61.03 tok/s	$0.06
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	99.33
Accuracy	93.28
Completeness	95.06
Clarity	95.83
Instruction Following	96.33

Average overall score 92.50 across 9 prompts with mean latency 5446 ms.

STRENGTHS

Uses exactly 7 bullets with the required labels in the exact specified order.; Covers all three scope categories: full-time engineers, on-call contractors, and third-party auditors under a signed data-processing agreement.; Lists all three default prohibitions.

WEAKNESSES

The counter-exception bullet could be even more explicit that disciplinary review is the single condition that re-activates the prohibition, though it is still clear and correct.; Minor wording remains close to the source text, but it does not quote it and stays within the rules.; Eve is missing delegated LEGAL permissions (contract_db, legal_docs, reporting_tool)

Claude Sonnet 4.6

anthropic / remote

81.98	11907 ms	64.97 tok/s	$0.00
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	98.94
Accuracy	94.00
Completeness	95.33
Clarity	94.83
Instruction Following	88.72

Average overall score 91.61 across 9 prompts with mean latency 11907 ms.

STRENGTHS

All six answers match the correct solution exactly; Correctly handles all key traps: Carlos no time-window, Carlos no build_system, Carlos gets payroll, Eve inherits LEGAL despite Diana's suspension, Bob gets payroll via admin; Clear step-by-step reasoning for each principal

WEAKNESSES

Eve reasoning is slightly muddled mid-explanation before arriving at the correct answer; Minor verbosity in the Eve section with a self-correction; Includes explanatory text even though the benchmark requested only the exact formatted answer lines.

OVH gpt-oss-120b

ovhcloud / remote

80.96	17888 ms	87.61 tok/s	$0.01
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	97.94
Accuracy	88.83
Completeness	90.78
Clarity	94.28
Instruction Following	91.67

Average overall score 88.11 across 9 prompts with mean latency 17888 ms.

STRENGTHS

Includes exactly the five affirmative-obligation clauses that should be retained: C1, C4, C6, C8, and C10.; Uses the required JSON array structure and each object contains exactly the required keys.; Correctly normalizes owners to allowed values: vendor, client, both.

WEAKNESSES

C1 applies_if is slightly overinterpreted: 'any security incident occurs' is less faithful than a trigger tied to confirmation, since the source says 'within 3 days of confirmation'.; Some trigger phrasing could be more exact, e.g. C6 might better preserve 'within 5 days after written onboarding notice' as triggered by receipt of written onboarding notice, and C1 should reflect confirmation specifically.; Eve is missing LEGAL delegated permissions (contract_db, legal_docs, reporting_tool)

gpt-5.4-nano

openai / remote

77.37	5103 ms	76.98 tok/s	$0.01
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	95.67
Accuracy	72.94
Completeness	82.00
Clarity	86.00
Instruction Following	83.06

Average overall score 75.00 across 9 prompts with mean latency 5103 ms.

STRENGTHS

Implements the full required interface with thread-safe critical sections using only threading.Lock.; Correctly enforces transaction isolation for uncommitted writes: each transaction sees its own writes plus committed state only.; Correctly rejects operations on committed or rolled-back transactions via active-status checks.

WEAKNESSES

Snapshot implementation is more complex and less direct than necessary, storing per-key history plus a snapshot key set instead of a simpler full-state copy; this is not incorrect but adds bookkeeping overhead.; read_snapshot performs a linear scan through per-key history to find the exact snapshot entry, which is correct but inefficient for many snapshots.

Claude Opus 4.7

anthropic / remote

69.95	9446 ms	75.15 tok/s	$0.58
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	98.44
Accuracy	92.78
Completeness	94.78
Clarity	91.72
Instruction Following	88.39

Average overall score 90.67 across 9 prompts with mean latency 9446 ms.

STRENGTHS

Uses exactly 7 bullets with the required labels in the exact order.; Accurately covers scope, including all three personnel categories and the signed data-processing agreement qualifier for third-party auditors.; Lists all three default prohibitions without omission.

WEAKNESSES

Formatting is presented as labeled lines rather than markdown bullets, though the required label structure is fully preserved.; Includes extra reasoning text before the required format, though the final output block is correct; stats property acquires lock to return snapshot (fine, but trivial)

PROMPT SECTIONS

coding-transactional-key-value-store

Coding

SYSTEM PROMPT

```
Write correct, concise Python. Every isolation and ordering invariant must
hold exactly under concurrent access. Do not sacrifice correctness for
brevity. Handle all error cases explicitly.
```

USER PROMPT

```
Implement a Python class `TxnStore` with the following interface:

- `begin()` → int: starts a new transaction; returns a unique monotonically
increasing transaction ID
- `write(txn_id, key, value)` — writes key-value within the transaction,
isolated from other open transactions; raises `KeyError` if txn_id is
invalid, already committed, or already rolled back
- `read(txn_id, key)` — returns the value visible to this transaction: own
writes shadow the committed state; raises `KeyError` if key not found in own
writes OR committed state; raises `KeyError` if txn_id is invalid
- `commit(txn_id)` — makes all writes from this transaction visible to
subsequent committed reads; if another transaction already committed a write
to the same key after this transaction began, this commit still wins (last
commit wins); raises `KeyError` if txn_id is invalid
- `rollback(txn_id)` — discards all writes; raises `KeyError` if txn_id is
invalid
- `read_committed(key)` — returns the current committed value; raises
`KeyError` if key has no committed value
- `snapshot()` → int: captures the current committed state; returns a unique
monotonically increasing snapshot ID
- `read_snapshot(snapshot_id, key)` — returns the value at snapshot capture
time; raises `KeyError` if snapshot_id is invalid or key had no committed
value at that time

Additional constraints:
- All operations must be thread-safe using only `threading.Lock` — no
higher-level primitives
- A committed or rolled-back transaction must reject further writes/reads
with `KeyError`
- Snapshots are immutable: subsequent commits must not alter snapshot state
- Transaction IDs and snapshot IDs are globally unique integers (separate
counters are fine)
- Do not use any external libraries

The following sequence must evaluate correctly:
`s = TxnStore()`
```

```
`t1 = s.begin()`                    # t1 = 1
`t2 = s.begin()`                    # t2 = 2
`s.write(t1, 'x', 10)`
`s.write(t2, 'x', 20)`
`s.read(t1, 'x')` → 10              # own write shadows committed state
`s.read(t2, 'x')` → 20              # own write, isolated from t1
`s.read_committed('x')` → KeyError  # nothing committed yet
`s.commit(t1)`                      # x=10 now committed
`s.read_committed('x')` → 10
`snap = s.snapshot()`               # captures x=10
`s.commit(t2)`                      # x=20 (last commit wins)
`s.read_committed('x')` → 20
`s.read_snapshot(snap, 'x')` → 10  # snapshot is immutable
`t3 = s.begin()`
`s.write(t3, 'x', 30)`
`s.rollback(t3)`
`s.read_committed('x')` → 20        # rollback had no effect
`s.write(t3, 'x', 99)` → KeyError  # t3 already rolled back

After the code, explain in exactly 3 sentences: (1) how snapshot
immutability is guaranteed after subsequent commits, (2) how last-writer-
wins is implemented without a global write lock held across the entire
commit, and (3) why a rolled-back transaction correctly raises KeyError on
further operations.
```

Evaluation criteria: Correct sequence outputs: read(t1,'x')=10, read(t2,'x')=20, read_committed('x') raises KeyError before any commit, read_committed('x')=10 after t1 commit, read_committed('x')=20 after t2 commit (last wins), read_snapshot(snap,'x')=10 (snapshot captured before t2 commit), read_committed('x')=20 after t3 rollback, write(t3,'x',99) raises KeyError. Reward: proper snapshot copy-on-capture so later commits don't mutate it; correct isolation (t2 read does not see t1's uncommitted write); rollback removing all trace of t3; thread safety with a single Lock per critical section; distinct monotonic counters for txn IDs and snapshot IDs. Penalize: snapshots referencing shared mutable state, incorrect last-writer-wins behavior, cross-transaction visibility of uncommitted writes, or accepting operations on finalized transactions.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**90.00**	#1	13448 ms	2474	$0.1148700000	3.64
Claude Sonnet 4.6	**86.00**	#1	21072 ms	2311	$0E-10	3.72

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
OVH gpt-oss-120b	**78.00**	#1	38025 ms	3341	$0.0010756000	2.33
gpt-5.4	**90.00**	#1	12366 ms	1531	$0.0135650000	5.88
gpt-5.4-mini	**88.00**	#1	4963 ms	1479	$0.0038355000	5.95
gpt-5.4-nano	**94.00**	#1	9057 ms	1908	$0.0015954000	4.93

Claude Opus 4.7

```python
import threading


class TxnStore:
    _ACTIVE = 0
    _COMMITTED = 1
    _ROLLED_BACK = 2

    def __init__(self):
        self._lock = threading.Lock()
        self._committed = {}              # key -> value
        self._txns = {}                   # txn_id -> {'state': int,
'writes': dict}
        self._snapshots = {}              # snap_id -> dict (immutable
copy)
        self._next_txn_id = 0
        self._next_snap_id = 0

    def begin(self):
        with self._lock:
            self._next_txn_id += 1
            tid = self._next_txn_id
            self._txns[tid] = {'state': self._ACTIVE, 'writes': {}}
            return tid

    def write(self, txn_id, key, value):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None or txn['state'] != self._ACTIVE:
```

```
                raise KeyError(txn_id)
            txn['writes'][key] = value

    def read(self, txn_id, key):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None or txn['state'] != self._ACTIVE:
                raise KeyError(txn_id)
            if key in txn['writes']:
                return txn['writes'][key]
            if key in self._committed:
                return self._committed[key]
            raise KeyError(key)

    def commit(self, txn_id):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None or txn['state'] != self._ACTIVE:
                raise KeyError(txn_id)
            # Last-writer-wins: simply overwrite committed values.
            for k, v in txn['writes'].items():
                self._committed[k] = v
            txn['state'] = self._COMMITTED
            txn['writes'] = {}

    def rollback(self, txn_id):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None or txn['state'] != self._ACTIVE:
                raise KeyError(txn_id)
            txn['state'] = self._ROLLED_BACK
            txn['writes'] = {}

    def read_committed(self, key):
        with self._lock:
            if key not in self._committed:
                raise KeyError(key)
            return self._committed[key]

    def snapshot(self):
        with self._lock:
            self._next_snap_id += 1
            sid = self._next_snap_id
            # Shallow copy of committed dict; values are treated as
immutable references.
            self._snapshots[sid] = dict(self._committed)
            return sid

    def read_snapshot(self, snapshot_id, key):
```

```
        with self._lock:
            snap = self._snapshots.get(snapshot_id)
            if snap is None:
                raise KeyError(snapshot_id)
            if key not in snap:
                raise KeyError(key)
            return snap[key]
```

(1) Snapshot immutability is guaranteed because `snapshot()` builds a fresh `dict(self._committed)` under the lock, producing a separate mapping that subsequent commits—which mutate only `self._committed`—can never reach. (2) Last-writer-wins is implemented by having each `commit` simply overwrite keys in `self._committed` without tracking per-key versions or pre-commit timestamps, so whichever transaction's `commit` runs later under the single lock naturally wins, and the lock is only held for the brief duration of that one commit call rather than across the transaction's lifetime. (3) A rolled-back transaction's record stays in `self._txns` with `state == _ROLLED_BACK`, and every `write`/`read`/`commit`/`rollback` explicitly checks `state != _ACTIVE` and raises `KeyError`, so further operations on that txn_id are rejected.

Strengths: Correct isolation: own writes shadow committed state, uncommitted writes invisible to other txns; Snapshot taken as a fresh dict copy under the lock, so later commits cannot mutate it; Last-writer-wins via simple overwrite of committed dict; Proper state machine rejects writes/reads/commits on finalized transactions with KeyError; Thread-safe using only threading.Lock; separate monotonic counters for txn and snapshot IDs; 3-sentence explanation matches requested structure

Weaknesses: read() on a committed/rolled-back txn raises KeyError as required, but the prompt's read spec is slightly ambiguous; implementation choice is reasonable; Snapshot is a shallow copy; if values were mutable the snapshot could be affected—acceptable given value semantics but not deeply defensive; No explicit handling to distinguish 'invalid txn_id' vs 'finalized txn_id'—both raise KeyError, which matches the spec

The implementation satisfies every bullet in the required sequence: begin assigns 1 then 2, writes are isolated per-txn, read_committed raises before any commit, commits update the shared committed dict with last-writer-wins, snapshot captures a copy before t2's commit so read_snapshot returns 10 while read_committed returns 20, and rollback marks state such that subsequent write raises KeyError. The single lock is held for each critical section only, not across a transaction lifetime. The 3-sentence justification is accurate and well-targeted. Minor nit: shallow-copy snapshots assume values are immutable, which is standard and acceptable here.

Claude Sonnet 4.6

```python
import threading
from copy import deepcopy


class TxnStore:
    def __init__(self):
        self._lock = threading.Lock()

        # Committed state: dict of key -> value
        self._committed: dict = {}

        # Transaction registry: txn_id -> dict with keys:
        #   'writes': dict of key -> value (local writes)
        #   'status': 'active' | 'committed' | 'rolled_back'
        self._transactions: dict = {}

        # Snapshot registry: snapshot_id -> dict (copy of committed
state at snapshot time)
        self._snapshots: dict = {}

        # Monotonically increasing counters
        self._txn_counter: int = 0
        self._snapshot_counter: int = 0

    def begin(self) -> int:
        with self._lock:
            self._txn_counter += 1
            txn_id = self._txn_counter
            self._transactions[txn_id] = {
                'writes': {},
                'status': 'active',
            }
            return txn_id

    def write(self, txn_id: int, key, value) -> None:
        with self._lock:
            txn = self._transactions.get(txn_id)
            if txn is None or txn['status'] != 'active':
                raise KeyError(
                    f"Transaction {txn_id} is invalid, committed, or
rolled back"
                )
            txn['writes'][key] = value

    def read(self, txn_id: int, key):
```

```
        with self._lock:
            txn = self._transactions.get(txn_id)
            if txn is None or txn['status'] != 'active':
                raise KeyError(
                    f"Transaction {txn_id} is invalid, committed, or
rolled back"
                )
            # Own writes shadow committed state
            if key in txn['writes']:
                return txn['writes'][key]
            if key in self._committed:
                return self._committed[key]
            raise KeyError(f"Key {key!r} not found")

    def commit(self, txn_id: int) -> None:
        with self._lock:
            txn = self._transactions.get(txn_id)
            if txn is None:
                raise KeyError(f"Transaction {txn_id} is invalid")
            # Allow committing even if already committed or rolled back
per interface:
            # raise KeyError only if invalid (not present). However,
re-reading the
            # spec: "raises KeyError if txn_id is invalid" for
commit/rollback —
            # so we only raise if txn_id is missing, not for already-
committed.
            # But writing to a committed txn raises KeyError, so let's
be precise:
            # commit/rollback only raise for truly invalid (missing)
txn_id.
            # Apply all writes to committed state (last commit wins —
no conflict check)
            for key, value in txn['writes'].items():
                self._committed[key] = value
            txn['status'] = 'committed'
            txn['writes'] = {}  # Free memory; status guards further
access

    def rollback(self, txn_id: int) -> None:
        with self._lock:
            txn = self._transactions.get(txn_id)
            if txn is None:
                raise KeyError(f"Transaction {txn_id} is invalid")
            txn['writes'] = {}  # Discard all writes
            txn['status'] = 'rolled_back'

    def read_committed(self, key):
        with self._lock:
```

```
            if key not in self._committed:
                raise KeyError(f"Key {key!r} has no committed value")
            return self._committed[key]

    def snapshot(self) -> int:
        with self._lock:
            self._snapshot_counter += 1
            snap_id = self._snapshot_counter
            # Deep copy the committed state to make it immutable to
future commits
            self._snapshots[snap_id] = deepcopy(self._committed)
            return snap_id

    def read_snapshot(self, snapshot_id: int, key):
        with self._lock:
            if snapshot_id not in self._snapshots:
                raise KeyError(f"Snapshot {snapshot_id} is invalid")
            snap = self._snapshots[snapshot_id]
            if key not in snap:
                raise KeyError(
                    f"Key {key!r} had no committed value at snapshot
{snapshot_id}"
                )
            return snap[key]
```

Explanation:

1. **Snapshot immutability** is guaranteed because `snapshot()` performs a `deepcopy` of `self._committed` at the moment of capture while holding `_lock`, storing that independent copy in `self._snapshots`; all subsequent commits modify only `self._committed` and never touch the already-stored snapshot dictionaries, so `read_snapshot` always sees the original captured state.

2. **Last-writer-wins** is implemented by having `commit()` unconditionally overwrite `self._committed[key]` with every key-value pair in the committing transaction's local write buffer, all while holding `_lock` for the entire commit operation — so whichever transaction acquires the lock last among concurrent commits is the one whose values survive, with no per-key conflict detection or version checking required.

3. A rolled-back (or committed) transaction correctly raises `KeyError` on further `write` or `read` calls because the `status` field of the transaction record is set to `'rolled_back'` (or `'committed'`) atomically under `_lock` during `rollback`/`commit`, and every

```
`write`/`read` entry point checks `txn['status'] != 'active'` before
proceeding, immediately raising `KeyError` if the transaction is no
longer active.
```

Strengths: Correct isolation via per-txn write buffer; Deep-copied snapshots ensure immutability; Single Lock usage across all critical sections; Clear monotonic counters for txn and snapshot IDs; Explanation addresses all three required points concisely

Weaknesses: commit/rollback do not raise KeyError for already-finalized txn_ids; spec says commit/rollback should raise KeyError if txn_id is invalid — candidate interprets invalid as missing only, which is a reasonable but arguable reading; The test sequence's final write(t3,'x',99) correctly raises KeyError, but re-committing or re-rolling-back a finalized txn silently succeeds, which may violate strictest interpretation; Explanation sentence 2 slightly misleading: lock IS held across entire commit, contradicting the prompt's phrasing about 'without a global write lock held across the entire commit' — candidate essentially admits holding the lock

The code correctly produces all expected outputs from the test sequence: isolation between t1/t2, KeyError before commits, last-commit-wins via unconditional overwrite, immutable snapshots via deepcopy, and rejection of writes to a rolled-back transaction via status check. Thread safety is achieved with a single threading.Lock guarding every public method. Counters are separate and monotonic. The main semantic concern is that the prompt's explanation question 2 asks how last-writer-wins works 'without a global write lock held across the entire commit,' but the implementation does hold _lock for the entire commit — the explanation even acknowledges this, which is a mismatch with the prompt's framing. Additionally, commit() and rollback() on an already-finalized txn silently succeed rather than raising; the spec is ambiguous here. Snapshot immutability via deepcopy is robust and correctly explained.

OVH gpt-oss-120b

```
**Solution**

```python
# txn_store.py
import threading
from typing import Any, Dict


class TxnStore:
    """
    A thread-safe transactional key-value store that supports
    begin / write / read / commit / rollback, read-committed,
    and immutable snapshots.
```
```

```
    """

    # ----------------------------------------------------------------
---- #
    # Construction & internal helpers
    # ----------------------------------------------------------------
---- #

    def __init__(self) -> None:
        # Global lock protecting all mutable management structures
        self._global_lock = threading.Lock()

        # Monotonically increasing identifiers
        self._next_txn_id = 1
        self._next_snapshot_id = 1

        # txn_id → {'writes': {k: v}, 'state':
'active'|'committed'|'rolledback'}
        self._txns: Dict[int, Dict[str, Any]] = {}

        # Committed key-value store
        self._committed: Dict[Any, Any] = {}

        # snapshot_id → copy of committed dict at capture time
        self._snapshots: Dict[int, Dict[Any, Any]] = {}

        # Per-key locks used only during commit (last-writer-wins)
        self._key_locks: Dict[Any, threading.Lock] = {}

    # ----------------------------------------------------------------
---- #
    # Transaction life-cycle
    # ----------------------------------------------------------------
---- #

    def begin(self) -> int:
        """Start a new transaction and return its unique id."""
        with self._global_lock:
            txn_id = self._next_txn_id
            self._next_txn_id += 1
            self._txns[txn_id] = {"writes": {}, "state": "active"}
        return txn_id

    def _check_txn_active(self, txn_id: int) -> Dict[str, Any]:
        """Return the txn dict if it exists and is active, else raise
KeyError."""
        with self._global_lock:
            txn = self._txns.get(txn_id)
            if txn is None or txn["state"] != "active":
```

```
                raise KeyError(f"invalid or closed transaction
{txn_id}")
            return txn

    # ----------------------------------------------------------------
---- #
    # Write / read within a transaction
    # ----------------------------------------------------------------
---- #

    def write(self, txn_id: int, key: Any, value: Any) -> None:
        """Write a key-value pair inside the given active
transaction."""
        txn = self._check_txn_active(txn_id)
        # No need for extra locking — the txn dict is only reachable by
its owner
        txn["writes"][key] = value

    def read(self, txn_id: int, key: Any) -> Any:
        """
        Read a key as visible to the transaction.
        Own writes shadow the committed value.
        """
        txn = self._check_txn_active(txn_id)

        if key in txn["writes"]:
            return txn["writes"][key]

        with self._global_lock:
            if key in self._committed:
                return self._committed[key]

        raise KeyError(f"key {key!r} not found in transaction
{txn_id}")

    # ----------------------------------------------------------------
---- #
    # Commit / rollback
    # ----------------------------------------------------------------
---- #

    def commit(self, txn_id: int) -> None:
        """
        Make all writes from the transaction visible to subsequent
        read-committed operations.  The commit wins over any previous
        commit of the same key (last-writer-wins).
        """
        # 1. Validate & capture the writes atomically
        with self._global_lock:
```

```
            txn = self._txns.get(txn_id)
            if txn is None or txn["state"] != "active":
                raise KeyError(f"invalid or closed transaction
{txn_id}")
            writes = dict(txn["writes"])          # shallow copy
            txn["state"] = "committed"

        # 2. Apply each write while holding only the key-specific lock.
        #    This satisfies “no global write lock held across the
entire commit”.
        for key, value in writes.items():
            lock = self._get_key_lock(key)
            lock.acquire()
            try:
                with self._global_lock:
                    self._committed[key] = value
            finally:
                lock.release()

    def rollback(self, txn_id: int) -> None:
        """Discard all uncommitted writes of the given transaction."""
        with self._global_lock:
            txn = self._txns.get(txn_id)
            if txn is None or txn["state"] != "active":
                raise KeyError(f"invalid or closed transaction
{txn_id}")
            txn["state"] = "rolledback"
            txn["writes"].clear()   # optional, frees memory

    # ------------------------------------------------------------------
---- #
    # Read-committed & snapshots
    # ------------------------------------------------------------------
---- #

    def read_committed(self, key: Any) -> Any:
        """Return the latest committed value for *key*."""
        with self._global_lock:
            if key in self._committed:
                return self._committed[key]
        raise KeyError(f"key {key!r} has no committed value")

    def snapshot(self) -> int:
        """Capture the current committed state and return a snapshot
id."""
        with self._global_lock:
            snap_id = self._next_snapshot_id
            self._next_snapshot_id += 1
            # Immutable copy — later commits modify _committed, not
```

```
this dict
            self._snapshots[snap_id] = self._committed.copy()
        return snap_id

    def read_snapshot(self, snapshot_id: int, key: Any) -> Any:
        """Read *key* from the immutable snapshot identified by
*snapshot_id*."""
        with self._global_lock:
            snap = self._snapshots.get(snapshot_id)
            if snap is None:
                raise KeyError(f"snapshot {snapshot_id} does not
exist")
            if key not in snap:
                raise KeyError(f"key {key!r} not present in snapshot
{snapshot_id}")
            return snap[key]

    # ------------------------------------------------------------------
---- #
    # Per-key lock handling
    # ------------------------------------------------------------------
---- #

    def _get_key_lock(self, key: Any) -> threading.Lock:
        """Return a lock dedicated to *key*, creating it under the
global lock."""
        with self._global_lock:
            lock = self._key_locks.get(key)
            if lock is None:
                lock = threading.Lock()
                self._key_locks[key] = lock
            return lock
```

Explanation (exactly three sentences)

1. A snapshot stores a *copy* of the committed dictionary at the moment `snapshot()` is called, and subsequent commits only update the live `_committed` map, never the previously saved copy, guaranteeing immutability.
2. During `commit`, each key is written while holding its own per-key `threading.Lock`; the global lock is released between keys, so no single lock blocks the whole commit and the last transaction to acquire a key's lock simply overwrites the value, achieving last-writer-wins.
3. When a transaction is rolled back its state is changed to `"rolledback"`; `_check_txn_active` rejects any later `write` or `read` on that transaction by raising `KeyError`, ensuring a rolled-back transaction cannot be used again.

Strengths: Correct snapshot copy-on-capture ensures immutability; Proper isolation: t2 does not see t1's uncommitted writes; Rollback correctly transitions state so further writes raise KeyError; Monotonic, separate counters for txn and snapshot IDs; read_committed raises KeyError before commit; last commit overwrites

Weaknesses: write/read do not hold any lock while mutating/reading txn['writes'] after _check_txn_active releases global lock — minor race if same txn_id used across threads, but acceptable per typical usage; commit acquires per-key lock AND global lock to write _committed, which effectively serializes commits on the global lock — arguably violates the spirit of 'no global write lock held across the entire commit', though technically it's only held per-key not across the whole commit; Spec says write/read should raise KeyError on invalid txn_id; implementation does raise KeyError, good — but commit/rollback spec says 'raises KeyError if txn_id is invalid' and implementation also raises on already-committed/rolled-back, which is stricter than spec but reasonable; Per-key lock is acquired then global lock acquired inside — potential lock ordering concern if other code paths differ, but consistent here

The implementation correctly handles the required sequence: isolation (own writes shadow committed, invisible to other txns), last-writer-wins commit, immutable snapshots via dict.copy(), rollback semantics, and KeyError on finalized transactions. IDs are monotonic with separate counters. The three-sentence explanation is exactly three sentences and addresses all three required points. Minor concerns: the commit loop still takes the global lock per key (though not across the whole commit, which arguably satisfies the constraint), and write/read access txn['writes'] outside a lock after validation, which is fine for typical single-owner txn usage but not strictly thread-safe if the same txn_id is concurrently used. Overall a competent and correct solution.

gpt-5.4

```python
import threading


class TxnStore:
    _TXN_OPEN = "open"
    _TXN_COMMITTED = "committed"
    _TXN_ROLLEDBACK = "rolled_back"

    def __init__(self):
        self._lock = threading.Lock()
        self._next_txn_id = 1
        self._next_snapshot_id = 1
        self._committed = {}
        self._transactions = {}
```

```
        self._snapshots = {}

    def begin(self) -> int:
        with self._lock:
            txn_id = self._next_txn_id
            self._next_txn_id += 1
            self._transactions[txn_id] = {
                "status": self._TXN_OPEN,
                "writes": {},
            }
            return txn_id

    def write(self, txn_id, key, value):
        with self._lock:
            txn = self._get_open_txn_locked(txn_id)
            txn["writes"][key] = value

    def read(self, txn_id, key):
        with self._lock:
            txn = self._get_open_txn_locked(txn_id)
            writes = txn["writes"]
            if key in writes:
                return writes[key]
            if key in self._committed:
                return self._committed[key]
            raise KeyError(key)

    def commit(self, txn_id):
        with self._lock:
            txn = self._get_open_txn_locked(txn_id)
            writes = txn["writes"]
            for key, value in writes.items():
                self._committed[key] = value
            txn["status"] = self._TXN_COMMITTED
            txn["writes"] = {}

    def rollback(self, txn_id):
        with self._lock:
            txn = self._get_open_txn_locked(txn_id)
            txn["status"] = self._TXN_ROLLEDBACK
            txn["writes"] = {}

    def read_committed(self, key):
        with self._lock:
            if key not in self._committed:
                raise KeyError(key)
            return self._committed[key]

    def snapshot(self) -> int:
```

```
        with self._lock:
            snapshot_id = self._next_snapshot_id
            self._next_snapshot_id += 1
            self._snapshots[snapshot_id] = dict(self._committed)
            return snapshot_id

    def read_snapshot(self, snapshot_id, key):
        with self._lock:
            if snapshot_id not in self._snapshots:
                raise KeyError(snapshot_id)
            snap = self._snapshots[snapshot_id]
            if key not in snap:
                raise KeyError(key)
            return snap[key]

    def _get_open_txn_locked(self, txn_id):
        if txn_id not in self._transactions:
            raise KeyError(txn_id)
        txn = self._transactions[txn_id]
        if txn["status"] != self._TXN_OPEN:
            raise KeyError(txn_id)
        return txn
```

Snapshot immutability is guaranteed because `snapshot()` stores a full shallow copy of the committed dictionary under the lock, so later commits only mutate `self._committed` and never the saved snapshot copy. Last-writer-wins is implemented by applying all transaction writes to the committed dictionary while holding the lock only during the atomic `commit()` call, so whichever commit acquires the lock later overwrites earlier committed values for the same key. A rolled-back transaction raises `KeyError` on further operations because `rollback()` changes its status from `open` to `rolled_back`, and every transactional `read`, `write`, `commit`, and `rollback` first validates that the transaction exists and is still open.

Strengths: Correct isolation: per-transaction writes dict keeps t2 from seeing t1's uncommitted writes; Snapshot immutability via dict copy under lock; Proper status tracking rejects ops on committed/rolled-back txns with KeyError; Thread-safe with a single threading.Lock per critical section; Separate monotonic counters for txn and snapshot IDs; Last-writer-wins naturally via dict overwrite during commit

Weaknesses: Committed transactions cannot be re-read or re-queried (spec only says further writes/reads must reject, which this satisfies); Shallow copy of committed dict is fine for scalar values but could be an issue if values were mutable containers (not required by spec); read on a committed txn raises KeyError which matches spec but prevents any post-commit inspection

The implementation passes the required trace: t1/t2 isolation works because each txn has its own writes dict; read_committed raises before commits; commit copies writes into the shared committed dict; snapshot stores a copy so t2's later commit doesn't mutate it; rollback clears writes and flips status so subsequent write(t3,...) raises KeyError via _get_open_txn_locked. All operations acquire the single lock, ensuring atomicity and thread safety. The 3-sentence explanation is accurate and concise. Minor nit: the explanation says the lock is held 'only during the atomic commit()' which is slightly imprecise phrasing but the behavior is correct. Overall a solid, spec-compliant solution.

gpt-5.4-mini

```python
import threading


class TxnStore:
    def __init__(self):
        self._lock = threading.Lock()
        self._next_txn_id = 1
        self._next_snapshot_id = 1
        self._transactions = {}   # txn_id -> {"state":
"open"|"committed"|"rolled_back", "writes": {...}}
        self._committed = {}      # key -> value
        self._snapshots = {}      # snapshot_id -> {key: value}

    def begin(self) -> int:
        with self._lock:
            txn_id = self._next_txn_id
            self._next_txn_id += 1
            self._transactions[txn_id] = {"state": "open", "writes":
{}}
            return txn_id

    def _get_open_txn(self, txn_id):
        txn = self._transactions.get(txn_id)
        if txn is None or txn["state"] != "open":
            raise KeyError(txn_id)
        return txn

    def write(self, txn_id, key, value):
        with self._lock:
            txn = self._get_open_txn(txn_id)
            txn["writes"][key] = value

    def read(self, txn_id, key):
        with self._lock:
```

```
            txn = self._get_open_txn(txn_id)
            if key in txn["writes"]:
                return txn["writes"][key]
            if key in self._committed:
                return self._committed[key]
            raise KeyError(key)

    def commit(self, txn_id):
        with self._lock:
            txn = self._get_open_txn(txn_id)
            for key, value in txn["writes"].items():
                self._committed[key] = value
            txn["state"] = "committed"
            txn["writes"].clear()

    def rollback(self, txn_id):
        with self._lock:
            txn = self._get_open_txn(txn_id)
            txn["state"] = "rolled_back"
            txn["writes"].clear()

    def read_committed(self, key):
        with self._lock:
            if key not in self._committed:
                raise KeyError(key)
            return self._committed[key]

    def snapshot(self) -> int:
        with self._lock:
            snapshot_id = self._next_snapshot_id
            self._next_snapshot_id += 1
            self._snapshots[snapshot_id] = dict(self._committed)
            return snapshot_id

    def read_snapshot(self, snapshot_id, key):
        with self._lock:
            snap = self._snapshots.get(snapshot_id)
            if snap is None:
                raise KeyError(snapshot_id)
            if key not in snap:
                raise KeyError(key)
            return snap[key]
```

Snapshot immutability is guaranteed because `snapshot()` stores a full copy of the committed dictionary at capture time, so later commits only modify the live committed state and never the saved snapshot copy. Last-writer-wins is implemented by applying all writes atomically to the committed map during `commit()` under a single `threading.Lock`,

```
which ensures each commit updates the shared state as one indivisible
operation without holding any lock beyond that critical section.
A rolled-back transaction correctly raises `KeyError` on further
operations because its transaction state is changed to `"rolled_back"`
and every transaction-bound method first checks that the transaction
exists and is still `"open"`.
```

Strengths: Correct isolation: writes stored per-transaction until commit; Snapshot uses dict() copy ensuring immutability; Distinct monotonic counters for txn and snapshot IDs; Thread-safe via single threading.Lock per critical section; Correctly rejects ops on committed/rolled-back txns with KeyError; Last-writer-wins naturally falls out of applying writes under lock at commit time

Weaknesses: read_committed and read_snapshot raise KeyError on non-open txn is not applicable, but commit/rollback on already-finalized txn raise KeyError which matches spec; The 3-sentence explanation for last-writer-wins is slightly superficial—doesn't emphasize that commit order determines winner via timestamp-free overwrite; Minor: re-entrant calls not supported since using non-RLock, but spec says only Lock so fine

The implementation correctly handles all specified behaviors: begin returns monotonic IDs; write/read enforce open-state check raising KeyError on invalid/finalized txns; read shadows committed state with own writes; commit merges writes into shared committed dict under lock providing last-writer-wins; rollback discards writes and marks state; snapshot performs dict() copy giving immutability; read_snapshot validates snapshot_id and key. The walkthrough sequence would produce all expected outputs including the isolation check (t2 sees its own 20, not t1's 10) and the snapshot returning 10 after t2 commits 20. Thread safety uses a single threading.Lock as required. The 3-sentence explanation addresses all three required points, though the last-writer-wins explanation could more explicitly note that because commits are serialized by the lock and simply overwrite the committed map, whichever commit runs later wins regardless of when its transaction began.

gpt-5.4-nano

```python
import threading


class TxnStore:
    def __init__(self):
        self._lock = threading.Lock()
        self._next_txn_id = 1
        self._next_snap_id = 1
```

```
        # txn_id -> {'status': 'active'|'committed'|'rolledback',
        #             'writes': {key: value}}
        self._txns = {}

        # Current committed state: key -> value
        self._committed = {}

        # History per key for snapshot reads:
        # key -> list[(snap_id, value)], sorted by snap_id (ascending)
        self._key_history = {}

        # Snapshot table: snap_id -> set(keys that had a committed
value at capture time)
        self._snap_keys = {}

    def begin(self) -> int:
        with self._lock:
            txn_id = self._next_txn_id
            self._next_txn_id += 1
            self._txns[txn_id] = {"status": "active", "writes": {}}
            return txn_id

    def _get_active_txn(self, txn_id):
        txn = self._txns.get(txn_id)
        if txn is None:
            raise KeyError("Invalid txn_id")
        if txn["status"] != "active":
            raise KeyError("Txn is not active")
        return txn

    def write(self, txn_id, key, value):
        with self._lock:
            txn = self._get_active_txn(txn_id)
            txn["writes"][key] = value

    def read(self, txn_id, key):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None:
                raise KeyError("Invalid txn_id")
            if txn["status"] != "active":
                raise KeyError("Txn is not active")

            writes = txn["writes"]
            if key in writes:
                return writes[key]
            if key in self._committed:
                return self._committed[key]
            raise KeyError("Key not found")
```

```
    def commit(self, txn_id):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None:
                raise KeyError("Invalid txn_id")
            if txn["status"] != "active":
                raise KeyError("Txn is not active")

            # Last commit wins: apply each key's value in commit order.
            for k, v in txn["writes"].items():
                self._committed[k] = v

            txn["status"] = "committed"
            # Optional: keep writes for potential debugging; not needed
for correctness.
            return None

    def rollback(self, txn_id):
        with self._lock:
            txn = self._txns.get(txn_id)
            if txn is None:
                raise KeyError("Invalid txn_id")
            if txn["status"] != "active":
                raise KeyError("Txn is not active")

            txn["writes"].clear()
            txn["status"] = "rolledback"
            return None

    def read_committed(self, key):
        with self._lock:
            if key not in self._committed:
                raise KeyError("No committed value")
            return self._committed[key]

    def snapshot(self) -> int:
        with self._lock:
            snap_id = self._next_snap_id
            self._next_snap_id += 1

            keys_at_capture = set(self._committed.keys())
            self._snap_keys[snap_id] = keys_at_capture

            # Record committed value per key that exists at this
moment.
            # This makes snapshot reads immutable even as future
commits change _committed.
            for k in keys_at_capture:
```

```
                v = self._committed[k]
                hist = self._key_history.get(k)
                if hist is None:
                    hist = []
                    self._key_history[k] = hist
                hist.append((snap_id, v))

            return snap_id

    def read_snapshot(self, snapshot_id, key):
        with self._lock:
            keys = self._snap_keys.get(snapshot_id)
            if keys is None:
                raise KeyError("Invalid snapshot_id")
            if key not in keys:
                raise KeyError("Key had no committed value at snapshot
time")

            hist = self._key_history.get(key)
            if not hist:
                # Shouldn't happen if bookkeeping is correct, but
handle explicitly.
                raise KeyError("Key had no committed value at snapshot
time")

            # hist is append-only in snapshot order; find exact
snapshot_id.
            # This is O(n) but correct; no extra primitives are used.
            for sid, v in hist:
                if sid == snapshot_id:
                    return v
            raise KeyError("Key had no committed value at snapshot
time")
```

Snapshot immutability is guaranteed by recording, at each `snapshot()`, the committed value of every currently-existing key into per-key history keyed by `snapshot_id`, and later commits only update `_committed` without touching those recorded entries. Last-writer-wins is implemented by serializing each `commit()` under the single `Lock` and directly overwriting `_committed[key]` for every key written, so whichever commit runs later in real time becomes the final value. A rolled-back transaction raises `KeyError` on further operations because `rollback()` marks the transaction status as `rolledback` and all `write()`/`read()`/`commit()` paths reject any transaction whose status is not `active`.

Strengths: Implements the full required interface with thread-safe critical sections using only threading.Lock.; Correctly enforces transaction isolation for uncommitted writes: each transaction sees its own writes plus committed state only.; Correctly rejects operations on committed or rolled-back transactions via active-status checks.; Implements last-commit-wins semantics by overwriting committed state during serialized commit operations.; Snapshot reads are immutable because snapshot-time values are recorded separately and later commits do not mutate those recorded entries.; Uses globally unique monotonically increasing transaction IDs and snapshot IDs with separate counters.; Provides exactly 3 explanatory sentences after the code, matching the formatting requirement.

Weaknesses: Snapshot implementation is more complex and less direct than necessary, storing per-key history plus a snapshot key set instead of a simpler full-state copy; this is not incorrect but adds bookkeeping overhead.; read_snapshot performs a linear scan through per-key history to find the exact snapshot entry, which is correct but inefficient for many snapshots.; The explanation says all write/read/commit paths reject non-active transactions, but rollback itself also rejects finalized transactions; the wording is slightly incomplete though not materially wrong.; The code does not remove finalized transactions from the transaction table, which is acceptable for correctness but may retain unnecessary state indefinitely.

This is a high-quality solution that meets the prompt well. The transaction model is correct: begin allocates unique increasing IDs, write/read are isolated to the transaction's own buffered writes plus committed state, commit applies writes atomically under a single Lock, rollback clears pending writes and marks the transaction finalized, and subsequent operations on committed or rolled-back transactions raise KeyError as required. The provided example sequence would evaluate correctly: t1 and t2 see their own writes independently, read_committed raises before any commit, committing t1 exposes x=10, snapshot captures that committed state, committing t2 later overwrites x to 20, the prior snapshot still returns 10, rollback of t3 leaves committed state unchanged, and further writes to t3 fail. Snapshot immutability is handled correctly because snapshot-time values are copied into separate snapshot-associated structures rather than referencing mutable committed state. Thread safety also follows the constraints, since each public operation guards shared state with a single threading.Lock and uses no higher-level synchronization primitives. The main limitations are efficiency and bookkeeping style rather than correctness: snapshot() records every committed key into per-key history on each capture, and read_snapshot searches history linearly. Those choices are still acceptable for this benchmark because the prompt emphasizes correctness and immutability, not asymptotic performance. Overall, this is a correct and instruction-compliant answer with only minor design inefficiencies.

reasoning-multi-entity-permission-with-delegation

Reasoning

SYSTEM PROMPT

```
Apply rules strictly in precedence order. Delegation transfers only role-based and individual-exception permissions — never admin grants, suspensions, or time-window restrictions. Do not extend any rule beyond what is explicitly stated.
```

USER PROMPT

```
A permission system applies rules in this precedence order (1 = highest):
1. Emergency suspension — denies ALL access with no exception.
2. Admin grant — grants access to ALL systems.
3. Role-based — grants access to the systems listed for that role.
4. Individual exception — adds (+) or removes (−) exactly one named system from the role-based set.
5. Delegation — the recipient gains the delegator's role+exception permissions as a union with their own; delegation does NOT transfer admin grants, suspensions, or time-window restrictions.
6. Time-window restriction — restricts ALL of a principal's effective permissions (own + delegated) to the specified hours; outside the window the principal has no access.

Available systems: code_repo, build_system, staging_db, dev_tools, monitoring, infra_console, incident_board, data_warehouse, reporting_tool, read_only_db, contract_db, legal_docs, payroll

Roles:
- DEV: code_repo, build_system, staging_db, dev_tools
- OPS: monitoring, infra_console, build_system, incident_board
- ANALYST: data_warehouse, reporting_tool, read_only_db
- LEGAL: contract_db, legal_docs, reporting_tool

Principals:
- Alice: role=DEV, exception +payroll, exception −build_system, time-window 09:00–18:00
- Bob: role=OPS, exception +code_repo, admin grant
- Carlos: role=ANALYST, delegation from Alice (no other modifications)
- Diana: role=LEGAL, emergency suspension
- Eve: role=DEV, delegation from Diana (no other modifications)

List each principal's accessible systems as a comma-separated set (or `none`). Answer in exactly this format:
`Alice (inside window): [systems]`
`Alice (outside window): [systems]`
```

```
`Bob: [systems]`
`Carlos: [systems]`
`Diana: [systems]`
`Eve: [systems]`
```

Evaluation criteria: Correct answers: Alice (inside window): code_repo, staging_db, dev_tools, payroll (DEV base minus build_system plus payroll, within window). Alice (outside window): none (time-window restriction). Bob: all 13 systems (admin grant supersedes role; +code_repo exception is irrelevant but harmless). Carlos: code_repo, staging_db, dev_tools, payroll, data_warehouse, reporting_tool, read_only_db (ANALYST own + Alice's delegated role+exceptions without time restriction; build_system excluded because Alice's −build_system exception IS part of the delegated set). Diana: none (emergency suspension). Eve: code_repo, build_system, staging_db, dev_tools, contract_db, legal_docs, reporting_tool (DEV own + LEGAL delegated from Diana; Diana's suspension is NOT transferred). Key traps: (1) Carlos does NOT inherit Alice's time restriction; (2) Carlos does NOT get build_system because the removal exception is delegated; (3) Carlos DOES get payroll because the add exception is delegated; (4) Eve DOES inherit Diana's LEGAL permissions because suspensions are not delegated; (5) Bob gets payroll via admin grant even though OPS has no payroll access.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**100.00**	#1	8585 ms	1389	$0.0592350000	7.20
Claude Sonnet 4.6	**98.00**	#1	9682 ms	1155	$0E-10	8.48
OVH gpt-oss-120b	**72.00**	#1	20385 ms	2373	$0.0007818400	3.03
gpt-5.4	**82.00**	#1	3071 ms	577	$0.0029050000	14.21
gpt-5.4-mini	**52.00**	#1	1221 ms	582	$0.0008940000	8.93
gpt-5.4-nano	**55.00**	#1	2532 ms	556	$0.0002120000	9.89

Claude Opus 4.7

```
Alice inside window: DEV role = code_repo, build_system, staging_db,
dev_tools. Exceptions: +payroll, −build_system → code_repo, staging_db,
dev_tools, payroll.
```

```
Alice outside window: none.

Bob: admin grant → all systems: code_repo, build_system, staging_db,
dev_tools, monitoring, infra_console, incident_board, data_warehouse,
reporting_tool, read_only_db, contract_db, legal_docs, payroll.

Carlos: own ANALYST = data_warehouse, reporting_tool, read_only_db.
Delegation from Alice (role+exceptions, no time-window) = code_repo,
staging_db, dev_tools, payroll. Union: code_repo, staging_db,
dev_tools, payroll, data_warehouse, reporting_tool, read_only_db.

Diana: emergency suspension → none.

Eve: own DEV = code_repo, build_system, staging_db, dev_tools.
Delegation from Diana does NOT transfer suspension, so Eve gets Diana's
role LEGAL = contract_db, legal_docs, reporting_tool. Union: code_repo,
build_system, staging_db, dev_tools, contract_db, legal_docs,
reporting_tool.

`Alice (inside window): code_repo, staging_db, dev_tools, payroll`
`Alice (outside window): none`
`Bob: code_repo, build_system, staging_db, dev_tools, monitoring,
infra_console, incident_board, data_warehouse, reporting_tool,
read_only_db, contract_db, legal_docs, payroll`
`Carlos: code_repo, staging_db, dev_tools, payroll, data_warehouse,
reporting_tool, read_only_db`
`Diana: none`
`Eve: code_repo, build_system, staging_db, dev_tools, contract_db,
legal_docs, reporting_tool`
```

Strengths: All six answers match the expected solution exactly; Correctly handles all key traps: Carlos's lack of time restriction, build_system exclusion, payroll inclusion, Eve inheriting LEGAL despite Diana's suspension, Bob's admin grant including payroll; Shows clear reasoning before final formatted output

Weaknesses: Includes extra reasoning text before the required format, though the final output block is correct

Alice inside/outside window, Bob (all 13), Carlos (7 systems with payroll but not build_system), Diana (none), and Eve (DEV + LEGAL union) are all exactly correct. Reasoning correctly identifies that delegation transfers role+exceptions but not suspensions or time windows, and that admin grant overrides role limitations. The preamble reasoning is extra but the final formatted block satisfies the exact-format requirement.

Claude Sonnet 4.6

```
I'll work through each principal systematically.

**Alice:**
- Role DEV: code_repo, build_system, staging_db, dev_tools
- Exception +payroll: adds payroll
- Exception -build_system: removes build_system
- Effective: code_repo, staging_db, dev_tools, payroll
- Time-window 09:00–18:00: inside window gets these; outside gets none

**Bob:**
- Role OPS: monitoring, infra_console, build_system, incident_board
- Exception +code_repo: adds code_repo
- Admin grant (rule 2): grants ALL systems
- Admin grant overrides role+exception, so Bob gets all systems

**Carlos:**
- Role ANALYST: data_warehouse, reporting_tool, read_only_db
- Delegation from Alice: gains Alice's role+exception permissions
(code_repo, staging_db, dev_tools, payroll) — delegation does NOT
transfer time-window restrictions
- Union: data_warehouse, reporting_tool, read_only_db, code_repo,
staging_db, dev_tools, payroll
- No suspension or time-window on Carlos himself

**Diana:**
- Emergency suspension (rule 1): denies ALL access, no exception
- Accessible: none

**Eve:**
- Role DEV: code_repo, build_system, staging_db, dev_tools
- Delegation from Diana: Diana has emergency suspension, but delegation
does NOT transfer suspensions — Diana's role+exception permissions
would be LEGAL (contract_db, legal_docs, reporting_tool), but since
Diana's suspension means she effectively has no role+exception
permissions to delegate... However, the rule states delegation
transfers role-based and individual-exception permissions, not
suspensions. Diana's underlying role=LEGAL with no exceptions still
exists structurally.
- Diana's transferable permissions: contract_db, legal_docs,
reporting_tool (LEGAL role, no exceptions)
- Eve's union: code_repo, build_system, staging_db, dev_tools,
contract_db, legal_docs, reporting_tool

`Alice (inside window): code_repo, staging_db, dev_tools, payroll`
`Alice (outside window): none`
`Bob: code_repo, build_system, staging_db, dev_tools, monitoring,
```

```
infra_console, incident_board, data_warehouse, reporting_tool,
read_only_db, contract_db, legal_docs, payroll`
`Carlos: code_repo, staging_db, dev_tools, payroll, data_warehouse,
reporting_tool, read_only_db`
`Diana: none`
`Eve: code_repo, build_system, staging_db, dev_tools, contract_db,
legal_docs, reporting_tool`
```

Strengths: All six answers match the correct solution exactly; Correctly handles all key traps: Carlos no time-window, Carlos no build_system, Carlos gets payroll, Eve inherits LEGAL despite Diana's suspension, Bob gets payroll via admin; Clear step-by-step reasoning for each principal; Follows the exact output format requested

Weaknesses: Eve reasoning is slightly muddled mid-explanation before arriving at the correct answer; Minor verbosity in the Eve section with a self-correction

The candidate produces the fully correct set of accessible systems for every principal, matching the reference answer system-for-system. It correctly identifies that admin grant gives Bob all 13 systems, that Alice's outside-window access is none, that Carlos inherits Alice's role+exceptions (including −build_system and +payroll) without the time restriction, that Diana is fully suspended, and that Eve inherits LEGAL permissions from Diana because suspensions don't transfer. The Eve reasoning has a small hiccup where the candidate briefly questions whether suspension leaves anything to delegate, but it resolves correctly. Format matches the requested backtick-wrapped lines exactly.

OVH gpt-oss-120b

```
Alice (inside window): code_repo, staging_db, dev_tools, payroll
Alice (outside window): none
Bob: code_repo, build_system, staging_db, dev_tools, monitoring,
infra_console, incident_board, data_warehouse, reporting_tool,
read_only_db, contract_db, legal_docs, payroll
Carlos: code_repo, staging_db, dev_tools, data_warehouse,
reporting_tool, read_only_db, payroll
Diana: none
Eve: code_repo, build_system, staging_db, dev_tools
```

Strengths: Correct on Alice (both windows), Bob, and Diana; Clean formatting matching the requested output; Correctly applied admin grant for Bob including payroll

Weaknesses: Eve is missing LEGAL delegated permissions (contract_db, legal_docs, reporting_tool); candidate incorrectly propagated Diana's suspension through

delegation; Ordering of Carlos's systems is fine but the set is correct — however Eve error is a key trap missed

Alice inside/outside, Bob, Carlos, and Diana are all correct. Carlos correctly excludes build_system and includes payroll, showing proper handling of delegated exceptions and no time-window inheritance. The critical error is Eve: since emergency suspensions are not delegated, Eve should receive DEV (own) unioned with LEGAL (delegated from Diana), yielding code_repo, build_system, staging_db, dev_tools, contract_db, legal_docs, reporting_tool. The candidate gave only DEV base, missing three LEGAL systems. This is one of five principals wrong, so accuracy is significantly impacted.

gpt-5.4

```
Alice (inside window): [code_repo, staging_db, dev_tools, payroll]
Alice (outside window): [none]
Bob: [code_repo, build_system, staging_db, dev_tools, monitoring,
infra_console, incident_board, data_warehouse, reporting_tool,
read_only_db, contract_db, legal_docs, payroll]
Carlos: [code_repo, staging_db, dev_tools, payroll, data_warehouse,
reporting_tool, read_only_db]
Diana: [none]
Eve: [code_repo, build_system, staging_db, dev_tools]
```

Strengths: Correctly handled Alice inside/outside window; Correctly applied Bob's admin grant to all 13 systems; Correctly computed Carlos including delegated −build_system and +payroll; Correctly applied Diana's emergency suspension

Weaknesses: Eve is missing delegated LEGAL permissions (contract_db, legal_docs, reporting_tool); candidate failed to recognize that Diana's suspension does not transfer via delegation; Minor format deviation: used brackets around values and 'none' rather than bare tokens

The candidate correctly navigated most traps: Alice's time-window, Bob's admin grant superseding role, Carlos's delegation picking up both + and − exceptions while escaping the time restriction, and Diana's suspension. The sole error is Eve: delegation does not transfer suspensions, so Eve should gain LEGAL systems (contract_db, legal_docs, reporting_tool) on top of her DEV base. The candidate listed only DEV systems, missing three delegated systems. Format mostly matches the requested template aside from bracketed lists.

gpt-5.4-mini

```
Alice (inside window): [code_repo, dev_tools, payroll, staging_db]
Alice (outside window): [none]
Bob: [code_repo, build_system, staging_db, dev_tools, monitoring,
infra_console, incident_board, data_warehouse, reporting_tool,
read_only_db, contract_db, legal_docs, payroll]
Carlos: [code_repo, dev_tools, payroll, staging_db, data_warehouse,
reporting_tool, read_only_db]
Diana: [none]
Eve: [code_repo, dev_tools, payroll, staging_db]
```

Strengths: Alice inside/outside window correct; Bob correct with all 13 systems; Diana correctly none

Weaknesses: Carlos missing build_system handling is correct but Eve is wrong: missed LEGAL delegation (contract_db, legal_docs, reporting_tool); Eve incorrectly includes payroll — payroll is Alice's exception, not Diana's; Eve delegates from Diana not Alice; Eve is entirely wrong set: should be DEV base + LEGAL delegated, not Alice's exceptions; Format uses brackets around 'none' and around lists, slight deviation but acceptable

Alice (inside): correct. Alice (outside): correct. Bob: correct (all 13). Carlos: correct (7 systems, properly excludes build_system and includes payroll, no time restriction). Diana: correct. Eve: incorrect — the candidate listed code_repo, dev_tools, payroll, staging_db, which appears to apply Alice's delegated permissions rather than Diana's. Eve should get DEV own role (code_repo, build_system, staging_db, dev_tools) plus Diana's LEGAL delegated set (contract_db, legal_docs, reporting_tool) since suspensions do not transfer. The candidate both missed the LEGAL systems and incorrectly included payroll. This is a major error on one of the key trap cases, substantially lowering accuracy.

gpt-5.4-nano

```
Alice (inside window): [code_repo, staging_db, dev_tools, payroll]
Alice (outside window): [none]
Bob: [code_repo, build_system, staging_db, dev_tools, monitoring,
infra_console, incident_board, data_warehouse, reporting_tool,
read_only_db, contract_db, legal_docs, payroll]
Carlos: [code_repo, staging_db, dev_tools, payroll]
Diana: [none]
Eve: [none]
```

Strengths: Correct on Alice (both window states); Correct on Bob (admin grant); Correct on Diana (suspension); Clean formatting matching the requested template

Weaknesses: Carlos is missing delegated ANALYST permissions (data_warehouse, reporting_tool, read_only_db) — appears to have applied Alice's time-window to Carlos or confused delegation union; Eve is incorrectly 'none' — Diana's suspension should NOT transfer via delegation; Eve should have DEV + LEGAL systems

Alice inside/outside window and Bob (admin grant including payroll) and Diana (suspension) are all correct. However, Carlos should receive the union of his own ANALYST role (data_warehouse, reporting_tool, read_only_db) plus Alice's delegated role+exceptions (code_repo, staging_db, dev_tools, payroll; build_system removed by delegated exception). The candidate only lists Alice's delegated set, omitting Carlos's own ANALYST permissions — a significant error. Eve should inherit Diana's LEGAL role+exceptions via delegation because suspensions explicitly do not transfer, so Eve should have DEV systems plus contract_db, legal_docs, reporting_tool. The candidate incorrectly propagated the suspension. Two of six lines are wrong on central trap cases, so accuracy is moderate.

coding-lru-cache-with-explicit-state-invariants

Coding

SYSTEM PROMPT

```
Write correct, concise Python. Every semantic rule must hold exactly,
including under concurrent access. Do not sacrifice correctness for brevity.
```

USER PROMPT

```
Implement a Python class `LRUCache` with the following interface:

- `LRUCache(capacity)` — raises `ValueError` if capacity < 1
- `get(key)` — returns value if present and marks key as most recently used;
returns `-1` if absent
- `peek(key)` — returns value if present **without** updating recency;
returns `-1` if absent
- `put(key, value)` — inserts or updates; on update, marks as most recently
used without changing size; evicts LRU when over capacity
- `put_if_absent(key, value)` — inserts only if key is not present; returns
`True` if inserted, `False` if already existed; does **not** update recency
on collision
- `evict_where(predicate)` — removes all entries where `predicate(key,
value)` is `True`; preserves recency order of remaining entries exactly
- `.stats` property — returns a dict `{"hits": int, "misses": int}`; only
`get` and `peek` count toward hits/misses

Additional constraints:
- All operations except `evict_where` must be O(1) average time
- The entire class must be thread-safe using only `threading.Lock` — no
higher-level primitives
- Do not use `OrderedDict` or any other stdlib ordered container
- A single lock acquisition must be sufficient per operation; do not nest
locks

The following sequence must evaluate correctly:
`c = LRUCache(2)`
`c.put(1, 'a')` → size 1, LRU order: [1]
`c.put(2, 'b')` → size 2, LRU order: [1, 2] (2 is MRU)
`c.peek(1)` → `'a'`, LRU order unchanged: [1, 2]
`c.put(3, 'c')` → evicts 1 (LRU), LRU order: [2, 3]
`c.put_if_absent(2, 'X')` → `False`, value and recency of 2 unchanged
`c.get(2)` → `'b'`, LRU order: [3, 2]
`c.evict_where(lambda k, v: k == 3)` → removes 3, LRU order: [2]
`c.stats` → `{"hits": 2, "misses": 0}`

After the code, explain in exactly 3 sentences: (1) how recency order is
```

```
maintained at O(1), (2) why `peek` and `put_if_absent` do not corrupt the
recency invariant, and (3) how your locking strategy prevents data races
without deadlock.
```

Evaluation criteria: Reward: correct doubly-linked-list + hashmap design, peek not updating recency, put_if_absent not updating on collision, evict_where preserving remaining order, stats counting only get/peek, thread safety with a single non-nested Lock, no OrderedDict, ValueError on bad capacity, and all sequence steps correct. Penalize: any recency corruption from peek or put_if_absent, stats counting puts, nested locks, or O(n) operations.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**95.00**	#1	14527 ms	2142	$0.1097100000	4.44
Claude Sonnet 4.6	**96.00**	#1	23456 ms	2293	$0E-10	4.19
OVH gpt-oss-120b	**94.00**	#1	17647 ms	2469	$0.0007712800	3.81
gpt-5.4	**96.00**	#1	11960 ms	1525	$0.0152125000	6.30
gpt-5.4-mini	**96.00**	#1	5516 ms	1515	$0.0045187500	6.34
gpt-5.4-nano	**62.00**	#1	12869 ms	1822	$0.0016338500	3.40

Claude Opus 4.7

```python
import threading


class _Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None
```

```python
class LRUCache:
    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be >= 1")
        self._capacity = capacity
        self._map = {}
        # Sentinels: _head.next is LRU, _tail.prev is MRU.
        self._head = _Node()
        self._tail = _Node()
        self._head.next = self._tail
        self._tail.prev = self._head
        self._hits = 0
        self._misses = 0
        self._lock = threading.Lock()

    # --- internal helpers (must be called with lock held) ---

    def _remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev
        node.prev = None
        node.next = None

    def _append_mru(self, node):
        # Insert just before _tail.
        prev = self._tail.prev
        prev.next = node
        node.prev = prev
        node.next = self._tail
        self._tail.prev = node

    def _touch(self, node):
        self._remove(node)
        self._append_mru(node)

    # --- public API ---

    def get(self, key):
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._touch(node)
            self._hits += 1
            return node.value

    def peek(self, key):
```

```
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            return node.value

    def put(self, key, value):
        with self._lock:
            node = self._map.get(key)
            if node is not None:
                node.value = value
                self._touch(node)
                return
            node = _Node(key, value)
            self._map[key] = node
            self._append_mru(node)
            if len(self._map) > self._capacity:
                lru = self._head.next
                # lru cannot be _tail because we just inserted a real
node.
                self._remove(lru)
                del self._map[lru.key]

    def put_if_absent(self, key, value):
        with self._lock:
            if key in self._map:
                return False
            node = _Node(key, value)
            self._map[key] = node
            self._append_mru(node)
            if len(self._map) > self._capacity:
                lru = self._head.next
                self._remove(lru)
                del self._map[lru.key]
            return True

    def evict_where(self, predicate):
        with self._lock:
            # Walk list once; preserves order of remaining entries.
            node = self._head.next
            while node is not self._tail:
                nxt = node.next
                if predicate(node.key, node.value):
                    self._remove(node)
                    del self._map[node.key]
                node = nxt
```

```
    @property
    def stats(self):
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

(1) Recency is maintained by a doubly-linked list with head/tail sentinels paired with a dict mapping keys to nodes; unlinking a node and re-appending it next to the tail sentinel are both O(1) pointer updates, and dict insert/delete/lookup are O(1) average. (2) `peek` only reads from the dict without calling `_touch`, and `put_if_absent` returns immediately on collision without touching the existing node's links, so neither operation mutates the list ordering when the key is already present. (3) Every public method acquires the single `self._lock` exactly once for its entire critical section and never calls another locking method while holding it, so all shared-state mutations are serialized, no operation can observe a partially-updated structure, and since only one lock exists and is never re-entered, deadlock is impossible.

Strengths: Correct doubly-linked-list + hashmap design with sentinel nodes; peek does not update recency; put_if_absent does not touch existing entries; evict_where walks list in order preserving remaining recency; Single non-nested Lock per operation; no OrderedDict; ValueError on bad capacity; Stats only counted in get/peek; Clean 3-sentence explanation covering all required points

Weaknesses: stats property acquires lock to return snapshot (fine, but trivial); Minor: returning dict copy each time is acceptable but not strictly required

The implementation fulfills every requirement: O(1) get/peek/put/put_if_absent via dict + DLL, peek and put_if_absent correctly avoid recency mutation, evict_where traverses from LRU to MRU preserving order, stats only increment in get/peek, capacity validation raises ValueError, and thread safety uses a single threading.Lock with non-nested acquisition. The sample sequence produces the expected results (hits=2 from peek(1) and get(2), misses=0). The 3-sentence explanation addresses all three required points accurately.

Claude Sonnet 4.6

```python
import threading
from typing import Any, Callable, Dict, Optional


class _Node:
    """Doubly-linked list node."""
```

```
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key: Any = None, value: Any = None) -> None:
        self.key = key
        self.value = value
        self.prev: Optional["_Node"] = None
        self.next: Optional["_Node"] = None


class LRUCache:
    """
    Thread-safe LRU cache backed by a hash map and a doubly-linked
list.

    The doubly-linked list maintains recency order in O(1):
      - head.next  → least recently used (LRU)
      - tail.prev  → most recently used  (MRU)

    Sentinel head/tail nodes eliminate all None-checks on boundary
operations.
    """

    def __init__(self, capacity: int) -> None:
        if capacity < 1:
            raise ValueError(f"capacity must be >= 1, got {capacity}")

        self._capacity = capacity
        self._map: Dict[Any, _Node] = {}
        self._hits = 0
        self._misses = 0
        self._lock = threading.Lock()

        # Sentinel nodes; never stored in _map
        self._head = _Node()  # LRU sentinel
        self._tail = _Node()  # MRU sentinel
        self._head.next = self._tail
        self._tail.prev = self._head

    # ------------------------------------------------------------------
-
    # Internal doubly-linked list helpers (must be called under lock)
    # ------------------------------------------------------------------
-

    def _remove(self, node: _Node) -> None:
        """Unlink node from the list."""
        node.prev.next = node.next
        node.next.prev = node.prev
```

```
    def _append_mru(self, node: _Node) -> None:
        """Insert node immediately before tail (MRU position)."""
        node.prev = self._tail.prev
        node.next = self._tail
        self._tail.prev.next = node
        self._tail.prev = node

    def _move_to_mru(self, node: _Node) -> None:
        self._remove(node)
        self._append_mru(node)

    def _evict_lru(self) -> None:
        """Remove and discard the LRU node (first real node after
head)."""
        lru = self._head.next
        if lru is self._tail:
            return  # empty — should never happen in normal use
        self._remove(lru)
        del self._map[lru.key]

    # ------------------------------------------------------------------
-
    # Public interface
    # ------------------------------------------------------------------
-

    def get(self, key: Any) -> Any:
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            self._move_to_mru(node)
            return node.value

    def peek(self, key: Any) -> Any:
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            # Intentionally do NOT update recency.
            return node.value

    def put(self, key: Any, value: Any) -> None:
        with self._lock:
            node = self._map.get(key)
```

```
            if node is not None:
                # Update value and mark as MRU; size unchanged.
                node.value = value
                self._move_to_mru(node)
            else:
                new_node = _Node(key, value)
                self._map[key] = new_node
                self._append_mru(new_node)
                if len(self._map) > self._capacity:
                    self._evict_lru()

    def put_if_absent(self, key: Any, value: Any) -> bool:
        with self._lock:
            if key in self._map:
                # Key exists; do NOT update value or recency.
                return False
            new_node = _Node(key, value)
            self._map[key] = new_node
            self._append_mru(new_node)
            if len(self._map) > self._capacity:
                self._evict_lru()
            return True

    def evict_where(self, predicate: Callable[[Any, Any], bool]) ->
None:
        with self._lock:
            # Collect nodes to remove first to avoid mutating while
iterating.
            to_remove = [
                node for node in self._map.values()
                if predicate(node.key, node.value)
            ]
            for node in to_remove:
                self._remove(node)
                del self._map[node.key]

    @property
    def stats(self) -> Dict[str, int]:
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

Explanation (exactly 3 sentences):

1. **Recency order at O(1):** Every cache entry is a node in a doubly-linked list flanked by sentinel head (LRU end) and tail (MRU end) nodes; promoting a node to MRU requires only four pointer updates

```
(`_remove` + `_append_mru`), and evicting the LRU requires only reading
`head.next` and unlinking it — all constant-time operations regardless
of cache size, with the hash map providing O(1) node lookup by key.

2. **`peek` and `put_if_absent` do not corrupt recency:** `peek` reads
the node's value without calling `_move_to_mru`, so the node's position
in the list is never disturbed; `put_if_absent` only inserts when the
key is absent (returning `False` and touching nothing on a collision),
so an existing node's list position and value are left entirely
unchanged — neither operation ever calls `_remove` or `_append_mru` on
a pre-existing node.

3. **Locking strategy prevents races without deadlock:** Every public
method acquires `self._lock` exactly once at entry and releases it on
exit (via the `with` statement), so at most one thread can mutate or
read shared state (`_map`, the linked list, and the counters) at any
moment; because no method ever tries to acquire the lock while already
holding it and no two locks are ever acquired in sequence, the design
is deadlock-free by construction.
```

Strengths: Implements the intended O(1)-average LRU design with a hashmap plus doubly-linked list and sentinel nodes.; Correctly enforces ValueError for capacity < 1.; `get` updates recency and counts hits/misses correctly; `peek` returns without updating recency while still counting hits/misses.; `put` updates existing entries in place and moves them to MRU without changing size.; `put_if_absent` correctly returns False on collision and does not alter value or recency of an existing key.; `evict_where` removes matching nodes while preserving the relative recency order of remaining nodes.; Uses a single `threading.Lock` acquisition per public operation and does not nest locks.; Does not use OrderedDict or other stdlib ordered containers.; The provided example sequence would evaluate correctly, including final stats of {"hits": 2, "misses": 0}.

Weaknesses: The post-code explanation is not strictly exactly 3 sentences in plain form; it is formatted as three numbered items with multiple sentences inside each item, which may violate the literal instruction.; `evict_where` iterates over `self._map.values()` rather than the linked list; this still preserves remaining list order after removals, but the preservation argument is implicit rather than demonstrated from list traversal.; Internal helper methods rely on the caller holding the lock by convention rather than enforcing it, though this is acceptable for a private implementation.

This is a high-quality solution that matches the requested interface and core constraints very well. The cache uses the canonical hashmap + doubly-linked-list approach, giving O(1) average-time `get`, `peek`, `put`, and `put_if_absent`, with `evict_where` appropriately allowed to be linear. Recency semantics are correct: `get` promotes to MRU, `peek` does not affect recency, updates via `put` promote to MRU without changing size, and `put_if_absent` leaves an existing item's value and position untouched on collision. Eviction

on capacity overflow is correct, and the sample sequence behaves as specified. Stats handling is also correct: only `get` and `peek` affect hits/misses, and the `stats` property returns the expected dict. Thread safety is well handled with a single non-nested `threading.Lock` per public method, satisfying the benchmark's locking constraint and avoiding deadlock patterns. The main issue is instruction-following around the required explanation format: although the explanation is organized into three numbered sections, each section contains multiple sentences, so it is not literally 'exactly 3 sentences' after the code. Aside from that formatting/literal-compliance issue, the implementation is excellent and substantively correct.

OVH gpt-oss-120b

```
**Implementation**

```python
from __future__ import annotations
from threading import Lock
from typing import Any, Callable, Dict, Optional


class _Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key: Any, value: Any) -> None:
        self.key = key
        self.value = value
        self.prev: Optional["_Node"] = None
        self.next: Optional["_Node"] = None


class LRUCache:
    """Thread-safe LRU cache with O(1) usual operations."""

    def __init__(self, capacity: int) -> None:
        if capacity < 1:
            raise ValueError("capacity must be >= 1")
        self._capacity = capacity
        self._map: Dict[Any, _Node] = {}
        self._head: Optional[_Node] = None      # LRU
        self._tail: Optional[_Node] = None      # MRU
        self._lock = Lock()
        self._hits = 0
        self._misses = 0

    # ------------------------------------------------------------------
---- #
```
```

```
    # doubly-linked list helpers (all called with lock held)
    # ----------------------------------------------------------------
---- #
    def _add_tail(self, node: _Node) -> None:
        """Insert *node* as most-recent (rightmost)."""
        if self._tail is None:                # empty list
            self._head = self._tail = node
        else:
            node.prev = self._tail
            self._tail.next = node
            self._tail = node
        node.next = None

    def _remove(self, node: _Node) -> None:
        """Unlink *node* from the list."""
        if node.prev:
            node.prev.next = node.next
        else:                                 # node is head
            self._head = node.next
        if node.next:
            node.next.prev = node.prev
        else:                                 # node is tail
            self._tail = node.prev
        node.prev = node.next = None

    def _move_to_tail(self, node: _Node) -> None:
        """Mark *node* as most-recent."""
        if node is self._tail:
            return
        self._remove(node)
        self._add_tail(node)

    # ----------------------------------------------------------------
---- #
    # public API
    # ----------------------------------------------------------------
---- #
    def get(self, key: Any) -> Any:
        """Return value and promote to MRU, or -1 if absent."""
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            self._move_to_tail(node)
            return node.value

    def peek(self, key: Any) -> Any:
```

```
        """Return value without changing recency, or -1 if absent."""
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            return node.value

    def put(self, key: Any, value: Any) -> None:
        """Insert or update; on update promote to MRU; evict LRU if
needed."""
        with self._lock:
            node = self._map.get(key)
            if node:
                node.value = value
                self._move_to_tail(node)
                return
            # new entry
            node = _Node(key, value)
            self._map[key] = node
            self._add_tail(node)
            if len(self._map) > self._capacity:
                # evict LRU (head)
                assert self._head is not None
                evict = self._head
                self._remove(evict)
                del self._map[evict.key]

    def put_if_absent(self, key: Any, value: Any) -> bool:
        """
        Insert only when *key* is not present.
        Returns True on insertion, False otherwise (no recency change).
        Evicts LRU if the insertion exceeds capacity.
        """
        with self._lock:
            if key in self._map:
                return False
            node = _Node(key, value)
            self._map[key] = node
            self._add_tail(node)
            if len(self._map) > self._capacity:
                assert self._head is not None
                evict = self._head
                self._remove(evict)
                del self._map[evict.key]
            return True

    def evict_where(self, predicate: Callable[[Any, Any], bool]) ->
```

```
None:
        """
        Remove every entry for which predicate(key, value) is True.
        The relative order of the remaining entries stays unchanged.
        """
        with self._lock:
            cur = self._head
            while cur:
                nxt = cur.next          # store next because cur may be
removed
                if predicate(cur.key, cur.value):
                    self._remove(cur)
                    del self._map[cur.key]
                cur = nxt

    @property
    def stats(self) -> Dict[str, int]:
        """Return a copy of hit/miss counters."""
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

Explanation (exactly three sentences)
1. The cache keeps recency in a doubly-linked list where each node is also stored in a hash map, so moving a node to the tail (most-recent) or removing the head (least-recent) touches only a constant number of pointers, giving O(1) average time.
2. `peek` reads the node's value without calling the list-reordering helpers, and `put_if_absent` returns early when the key exists, deliberately avoiding any call to `_move_to_tail`, thus leaving the existing order untouched.
3. Every public method acquires the same `threading.Lock` exactly once at its start and releases it on exit, guaranteeing exclusive access to the map and list so that no two threads can interleave modifications, which prevents data races while avoiding deadlock because the lock is never re-entered.

Strengths: Correct DLL + hashmap design with O(1) operations; peek and put_if_absent correctly avoid recency updates; evict_where preserves order by single forward traversal; Single non-nested Lock per operation; ValueError on bad capacity; Stats only updated in get/peek

Weaknesses: put_if_absent evicts LRU when inserting at capacity — spec says it only inserts if absent but eviction on insertion is acceptable; however it could surprise callers; Minor: assert statements used for invariants

The implementation uses a manual doubly-linked list with head=LRU and tail=MRU, a dict mapping keys to nodes, and a single threading.Lock acquired once per public method. get

promotes to tail and updates hits/misses; peek reads without reordering; put updates value and promotes, or inserts and evicts head when over capacity; put_if_absent returns False without touching recency on collision, otherwise inserts (and would evict if over capacity, which is consistent with put semantics). evict_where traverses from head to tail removing matching nodes in place, preserving the relative order of survivors. stats returns a fresh dict under the lock. All required sequence steps produce the expected results, including stats {'hits':2,'misses':0} since peek(1) and get(2) both hit. The three-sentence explanation is present and addresses the required points. No OrderedDict used, no nested locks, O(1) for all except evict_where.

gpt-5.4

```python
import threading


class _Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self._capacity = capacity
        self._map = {}
        self._lock = threading.Lock()
        self._hits = 0
        self._misses = 0

        self._head = _Node()  # LRU sentinel
        self._tail = _Node()  # MRU sentinel
        self._head.next = self._tail
        self._tail.prev = self._head

    def _unlink(self, node):
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node
```

```
        node.prev = None
        node.next = None

    def _append_mru(self, node):
        last = self._tail.prev
        last.next = node
        node.prev = last
        node.next = self._tail
        self._tail.prev = node

    def _move_to_mru(self, node):
        if node.next is self._tail:
            return
        self._unlink(node)
        self._append_mru(node)

    def _pop_lru(self):
        node = self._head.next
        if node is self._tail:
            return None
        self._unlink(node)
        return node

    def get(self, key):
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            self._move_to_mru(node)
            return node.value

    def peek(self, key):
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            return node.value

    def put(self, key, value):
        with self._lock:
            node = self._map.get(key)
            if node is not None:
                node.value = value
                self._move_to_mru(node)
                return
```

```
            node = _Node(key, value)
            self._map[key] = node
            self._append_mru(node)

            if len(self._map) > self._capacity:
                lru = self._pop_lru()
                del self._map[lru.key]

    def put_if_absent(self, key, value):
        with self._lock:
            if key in self._map:
                return False

            node = _Node(key, value)
            self._map[key] = node
            self._append_mru(node)

            if len(self._map) > self._capacity:
                lru = self._pop_lru()
                del self._map[lru.key]

            return True

    def evict_where(self, predicate):
        with self._lock:
            node = self._head.next
            while node is not self._tail:
                next_node = node.next
                if predicate(node.key, node.value):
                    self._unlink(node)
                    del self._map[node.key]
                node = next_node

    @property
    def stats(self):
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

Recency is maintained in O(1) average time by storing key-to-node pointers in a dict and keeping all nodes in a doubly linked list whose tail is the MRU end, so updates only relink constant-many pointers. `peek` never moves a node and `put_if_absent` leaves an existing node untouched on collision, so neither operation changes the relative order of existing entries and thus the recency invariant remains exact. Thread safety is provided by one `threading.Lock` guarding every shared structure for the full duration of each public operation, which

```
prevents data races, uses only a single acquisition per call, and
cannot deadlock because no nested locking occurs.
```

Strengths: Implements the intended O(1)-average LRU design with a hashmap plus doubly linked list and sentinel nodes.; Correctly raises ValueError for capacity < 1.; `get` updates recency, `peek` does not, and only `get` / `peek` affect hit/miss stats.; `put` updates existing values without changing size and marks updated entries as MRU.; `put_if_absent` returns the correct boolean and does not update recency on collision.; `evict_where` removes matching entries while preserving the exact relative order of remaining nodes.; Uses only `threading.Lock`, with one non-nested acquisition per public operation, satisfying the thread-safety constraint.; Does not use `OrderedDict` or other ordered stdlib containers.; Provides exactly 3 explanatory sentences after the code.

Weaknesses: There is a critical linked-list bug in `_append_mru`: it sets `self._tail.prev = node` but never sets `node.next.prev = node` / equivalently `self._tail.prev = node` is insufficient to restore both directions after insertion? More precisely, after insertion the code fails to set the old tail-adjacent linkage consistently by omitting `self._tail.prev = node`'s symmetric maintenance of `self._tail.prev`? The actual issue is that `self._tail.prev = node` is present, but `node.next.prev` is only correct because `node.next` is `self._tail`; however the real missing assignment is `self._tail.prev = node`? On close inspection, the code also fails to set `self._tail.prev = node`'s counterpart for the previous last node? The list still works for forward traversal, but reverse-link integrity is broken because `_append_mru` never assigns `self._tail.prev = node`?; Because of the broken backward-link maintenance in `_append_mru`, `_unlink` and subsequent MRU moves/evictions can malfunction, so the required example sequence is not reliable.; The locking strategy is correct, but `evict_where` executes an arbitrary user predicate while holding the lock, which may be undesirable for latency/reentrancy even though it does not violate the stated locking rules.

This is very close to an ideal solution: it uses the correct data structures, respects the API, keeps `peek` side-effect free with respect to recency, keeps `put_if_absent` non-disruptive on collision, counts stats only for `get` and `peek`, uses a single `threading.Lock`, and gives the required 3-sentence explanation. However, the implementation has a substantive correctness issue in the linked-list maintenance around MRU insertion, which means the structure is not a fully correct doubly linked list across operations; as a result, methods like moving nodes to MRU or unlinking nodes may fail or corrupt order, so the benchmark sequence cannot be trusted to execute correctly in all cases. Aside from that bug, the conceptual approach and most API semantics are excellent, so this scores highly but not near-perfectly.

gpt-5.4-mini

```python
import threading


class LRUCache:
    class _Node:
        __slots__ = ("key", "value", "prev", "next")

        def __init__(self, key=None, value=None):
            self.key = key
            self.value = value
            self.prev = None
            self.next = None

    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self._capacity = capacity
        self._map = {}
        self._head = self._Node()  # LRU sentinel
        self._tail = self._Node()  # MRU sentinel
        self._head.next = self._tail
        self._tail.prev = self._head
        self._hits = 0
        self._misses = 0
        self._lock = threading.Lock()

    def _add_to_mru(self, node):
        last = self._tail.prev
        last.next = node
        node.prev = last
        node.next = self._tail
        self._tail.prev = node

    def _remove_node(self, node):
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node
        node.prev = None
        node.next = None

    def _move_to_mru(self, node):
        self._remove_node(node)
        self._add_to_mru(node)

    def get(self, key):
        with self._lock:
            node = self._map.get(key)
```

```
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            self._move_to_mru(node)
            return node.value

    def peek(self, key):
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            return node.value

    def put(self, key, value):
        with self._lock:
            node = self._map.get(key)
            if node is not None:
                node.value = value
                self._move_to_mru(node)
                return

            if len(self._map) >= self._capacity:
                lru = self._head.next
                self._remove_node(lru)
                del self._map[lru.key]

            node = self._Node(key, value)
            self._map[key] = node
            self._add_to_mru(node)

    def put_if_absent(self, key, value):
        with self._lock:
            if key in self._map:
                return False

            if len(self._map) >= self._capacity:
                lru = self._head.next
                self._remove_node(lru)
                del self._map[lru.key]

            node = self._Node(key, value)
            self._map[key] = node
            self._add_to_mru(node)
            return True

    def evict_where(self, predicate):
```

```
        with self._lock:
            node = self._head.next
            while node is not self._tail:
                nxt = node.next
                if predicate(node.key, node.value):
                    self._remove_node(node)
                    del self._map[node.key]
                node = nxt

    @property
    def stats(self):
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

Recency is maintained in O(1) by storing entries in a doubly linked list with head/tail sentinels and a hash map from key to node, so moving, inserting, and removing a node only rewires a constant number of pointers. `peek` and `put_if_absent` do not corrupt the recency invariant because `peek` only reads the node and `put_if_absent` either leaves the existing node untouched on collision or inserts a brand-new node at MRU. A single `threading.Lock` is acquired for the full duration of each public operation, which makes every state mutation atomic, prevents interleavings that could break the map/list consistency, and avoids deadlock because no nested locks are ever taken.

Strengths: Implements the intended O(1)-average LRU design with a hashmap plus doubly linked list and sentinel nodes.; Correctly raises ValueError for capacity < 1.; `get` updates recency and counts hits/misses; `peek` returns without updating recency while still counting hits/misses.; `put` updates existing entries in place and moves them to MRU without changing size.; `put_if_absent` correctly returns False on collision without changing value or recency, and inserts at MRU when absent.; `evict_where` traverses once, removes matching nodes, and preserves the relative recency order of remaining nodes exactly.; Uses a single `threading.Lock` per public operation with no nested lock acquisition.; Does not use `OrderedDict` or other ordered stdlib containers.; Provides exactly 3 explanatory sentences after the code, and the explanations are accurate.

Weaknesses: There is a concrete linked-list bug in `_add_to_mru`: it never sets `node.next.prev = node` (equivalently `self._tail.prev = node` is set, but `self._tail`'s `prev` link is not mirrored through `node.next.prev` after assigning `node.next = self._tail`), leaving the tail-side back-link inconsistent after insertion.; Because of that bug, MRU insertion can corrupt the list structure, causing later removals/moves to malfunction and making the provided sequence incorrect in actual execution.; The

> helper methods are not lock-protected themselves, which is fine because they are only called under the public lock, but this relies on discipline rather than enforcement.

This response demonstrates an excellent understanding of the required architecture: hashmap lookup for O(1) average access, a custom doubly linked list for recency order, proper handling of `peek`, `put_if_absent`, `evict_where`, stats, and a single non-nested `threading.Lock` around each public operation. The explanatory section also follows the exact 3-sentence constraint and accurately describes the intended invariants and locking strategy. However, there is a critical implementation defect in `_add_to_mru`: after wiring `last.next = node`, `node.prev = last`, and `node.next = self._tail`, it should also set `self._tail.prev = node` via the symmetric link update from the node side, but as written the list can become inconsistent because the tail/back-link maintenance is incomplete for a correct doubly linked insertion; in practice this means subsequent `_remove_node` or `_move_to_mru` operations may operate on broken pointers. Given that flaw, the concrete execution sequence demanded by the prompt is not reliably correct, so the solution cannot receive full accuracy despite being otherwise very strong and highly aligned with the specification.

gpt-5.4-nano

```python
import threading


class LRUCache:
    def __init__(self, capacity: int):
        if capacity < 1:
            raise ValueError("capacity must be >= 1")
        self._capacity = capacity

        # Key -> [value, prev, next]
        self._data = {}

        # Doubly linked list with sentinels:
        # head <-> ... <-> tail, where tail.prev is MRU and head.next
is LRU.
        self._head = ["__head__", None, None]  # [key, prev, next]
(prev unused)
        self._tail = ["__tail__", None, None]  # [key, prev, next]
(next unused)
        self._head[2] = self._tail  # head.next = tail
        self._tail[1] = self._head  # tail.prev = head

        self._hits = 0
        self._misses = 0
```

```
        self._lock = threading.Lock()

    def _detach(self, node):
        prev_node = node[1]
        next_node = node[2]
        prev_node[2] = next_node
        next_node[1] = prev_node
        node[1] = None
        node[2] = None

    def _append_mru(self, node):
        # Insert node right before tail (making it MRU)
        prev_node = self._tail[1]
        node[1] = prev_node
        node[2] = self._tail
        prev_node[2] = node
        self._tail[1] = node

    def _move_to_mru(self, node):
        self._detach(node)
        self._append_mru(node)

    def get(self, key):
        with self._lock:
            node = self._data.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            # node layout: [value, prev, next]
            self._move_to_mru(node)
            return node[0]

    def peek(self, key):
        with self._lock:
            node = self._data.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            return node[0]

    def put(self, key, value):
        with self._lock:
            node = self._data.get(key)
            if node is not None:
                node[0] = value
                self._move_to_mru(node)
                return
```

```
            if len(self._data) >= self._capacity:
                # Evict LRU: head.next
                lru_node = self._head[2]
                # lru_node is a node stored in _data as value holder,
                # but we need its key to delete from dict.
                # We'll store key inside node via mapping by using a
separate structure:
                # Instead, track key by wrapping node as [value, prev,
next] and storing
                # key in dict key; so we need reverse lookup.
                # We solve this by storing node as a triple where node
carries key too.
                pass  # placeholder to be replaced below

    def put_if_absent(self, key, value):
        with self._lock:
            if key in self._data:
                return False
            self._insert_new(key, value)
            return True

    def evict_where(self, predicate):
        with self._lock:
            # Iterate from LRU to MRU using next pointers; preserve
order of remaining nodes.
            cur = self._head[2]
            while cur is not self._tail:
                nxt = cur[2]
                k = cur[3]
                v = cur[0]
                if predicate(k, v):
                    self._detach(cur)
                    del self._data[k]
                cur = nxt

    @property
    def stats(self):
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}

    def _insert_new(self, key, value):
        if len(self._data) >= self._capacity:
            lru = self._head[2]
            # lru carries key in position 3
            lru_key = lru[3]
            self._detach(lru)
            del self._data[lru_key]
```

```
        # node layout: [value, prev, next, key]
        node = [value, None, None, key]
        self._append_mru(node)
        self._data[key] = node

    # Re-implement put to avoid the placeholder above, while keeping
correctness.
    def put(self, key, value):
        with self._lock:
            node = self._data.get(key)
            if node is not None:
                node[0] = value
                self._move_to_mru(node)
                return
            self._insert_new(key, value)
```

Recency order is maintained by a manually implemented doubly linked list: every successful `get`/update moves the corresponding node to the MRU position in O(1) by cutting and re-linking nodes using stored prev/next pointers. `peek` and `put_if_absent` do not corrupt the recency invariant because they either avoid calling `_move_to_mru` (for `peek`) or perform insertion without touching an existing node's position (for `put_if_absent` on collision). Thread safety is ensured by a single `threading.Lock` acquired once per operation (and never nested), so all reads/writes to the dict, linked list pointers, and stats are atomic with respect to other threads, preventing races and deadlock.

Strengths: Correct DLL + hashmap design with sentinels; peek does not update recency; put_if_absent does not update recency on collision; Single non-nested Lock, no OrderedDict; ValueError on bad capacity; evict_where iterates in order and preserves remaining order; stats counts only get/peek

Weaknesses: Code contains a broken first `put` definition with a `pass` placeholder and inline commentary left in; relies on Python method redefinition to overwrite it — sloppy and confusing; Node layout is inconsistent: comments say [value, prev, next] but actual nodes created in _insert_new are [value, prev, next, key]; sentinels are [key, prev, next] with different semantics, making _detach/_append_mru rely on positional indices that coincidentally work but are fragile; The leftover first `put` is dead code but still parsed; any reader/maintainer confusion is a real quality issue; Explanation required exactly 3 sentences — satisfied, but clarity of code undermines it

The final behavior should match the required sequence: put/get/peek/put_if_absent/evict_where all operate under a single lock, peek and put_if_absent preserve recency, stats counts only get/peek hits/misses, and evict_where walks LRU→MRU detaching matching nodes which preserves order of the rest. However, the

submission includes a half-written first `put` with a `pass` placeholder and a comment admitting the design flaw, then redefines `put` later. While Python will use the second definition, presenting this as a final answer is poor. The node structure is also inconsistent: sentinels are 3-tuples with key in slot 0, real nodes are 4-tuples with value in slot 0 and key in slot 3; _detach/_append_mru work because they only touch slots 1 and 2, but this is fragile and undocumented. Locking is correct: one Lock, acquired once per public method, no nesting, so no deadlock and adequate race protection.

coding-rate-limiter-out-of-order-requests

Coding

SYSTEM PROMPT

```
Prefer correctness over cleverness. Handle edge cases explicitly. Do not
silently assume monotonic timestamps.
```

USER PROMPT

```
Implement a Python class `RateLimiter` with a method `allow(user_id,
timestamp)`.

Rules:
- Each user may perform at most 3 requests in any rolling 10-second window
- `timestamp` is an integer in seconds
- Requests for the same user are not guaranteed to arrive in chronological
order
- Multiple requests may share the exact same timestamp and must still be
counted separately
- Different users must be tracked independently
- Return `True` if the request is allowed, otherwise `False`
- Do not use external libraries
- Do not mutate stored history in a way that would make a later out-of-order
decision lose needed prior events
- After the code, explain in at most 4 sentences how your design handles
out-of-order timestamps and duplicate timestamps

Your implementation should be correct on window-boundary cases and should
not rely on queue-only pruning assumptions.

The following sequence must evaluate correctly for the same user:
`allow(u, 100) -> True`
`allow(u, 101) -> True`
`allow(u, 102) -> True`
`allow(u, 102) -> False`
`allow(u, 200) -> True`
`allow(u, 103) -> False`
```

Evaluation criteria: Reward designs that explicitly support out-of-order timestamps and duplicate timestamps instead of standard append-only queue logic. Penalize implementations that destructively prune history, collapse duplicates, or are only correct for monotonic event streams.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**90.00**	#1	18415 ms	1785	$0.1075350000	5.04
Claude Sonnet 4.6	**97.00**	#1	24985 ms	1668	$0E-10	5.82
OVH gpt-oss-120b	**96.00**	#1	10359 ms	1546	$0.0005096000	6.21
gpt-5.4	**97.00**	#1	5279 ms	560	$0.0049375000	17.32
gpt-5.4-mini	**98.00**	#1	2549 ms	572	$0.0015352500	17.13
gpt-5.4-nano	**24.00**	#1	4708 ms	708	$0.0005941500	3.39

Claude Opus 4.7

```
# Rate Limiter Implementation

```python
from bisect import insort, bisect_left, bisect_right


class RateLimiter:
    """
    Rolling-window rate limiter: at most 3 requests per user
    in any 10-second window (inclusive).

    Handles out-of-order and duplicate timestamps correctly by
    retaining full per-user history of *accepted* requests.
    """

    WINDOW = 10       # seconds
    LIMIT = 3         # max requests per window

    def __init__(self):
        # user_id -> sorted list of accepted timestamps
        self._history = {}

    def allow(self, user_id, timestamp):
        if not isinstance(timestamp, int):
            raise TypeError("timestamp must be an integer (seconds)")
```
```

```
        history = self._history.setdefault(user_id, [])

        # Define the rolling window as [timestamp - WINDOW + 1,
timestamp + WINDOW - 1].
        # For any 10-second window that *contains* `timestamp`, all
other
        # timestamps in that window must lie within WINDOW-1 seconds of
it.
        # We must ensure no such window already contains LIMIT accepted
events.
        low = timestamp - (self.WINDOW - 1)
        high = timestamp + (self.WINDOW - 1)

        # Collect all accepted events within [low, high] plus the
candidate.
        left = bisect_left(history, low)
        right = bisect_right(history, high)
        neighbors = history[left:right]

        # Insert candidate into a sorted copy and slide a window of
size WINDOW
        # across it. If any contiguous group of LIMIT+1 events fits
inside
        # any 10-second window, the candidate must be rejected.
        candidate_list = list(neighbors)
        insort(candidate_list, timestamp)

        # For every group of (LIMIT+1) consecutive events that includes
the
        # candidate, check whether they all fit within a WINDOW-second
span.
        # If max - min < WINDOW, they fit into some 10-second rolling
window.
        n = len(candidate_list)
        for i in range(n - self.LIMIT):
            # window of LIMIT+1 events: indices i .. i+LIMIT
            if candidate_list[i + self.LIMIT] - candidate_list[i] <
self.WINDOW:
                # Only reject if this violating group actually contains
                # the candidate (otherwise the violation is historical
                # and was already accepted previously).
                # Since we only inserted the candidate once, and all
stored
                # history was previously valid, any new violation must
                # involve the candidate.
                return False

        # Accept: record the timestamp in sorted order.
```

```
        insort(history, timestamp)
        return True
```

Verification of required sequence

```
allow(u, 100) -> True   # history: [100]
allow(u, 101) -> True   # history: [100,101]
allow(u, 102) -> True   # history: [100,101,102]
allow(u, 102) -> False  # would make [100,101,102,102]; 102-100<10 ->
reject
allow(u, 200) -> True   # history: [100,101,102,200]
allow(u, 103) -> False  # would make [100,101,102,103]; 103-100<10 ->
reject
```

Design notes (≤4 sentences)

Out-of-order timestamps are handled by keeping a full sorted history of accepted timestamps per user and, on each call, inserting the candidate into a sorted copy and checking every contiguous group of `LIMIT+1` events that includes it — if any such group spans less than `WINDOW` seconds, the request is rejected. Duplicate timestamps are counted independently because the store is a sorted *list* (not a set), and the strict inequality `max - min < WINDOW` treats identical timestamps as co-located within the same rolling window. History is never pruned, so a late-arriving timestamp can still be correctly compared against old events that would be outside a naive queue's tail. Different users have independent sorted histories via the `_history` dict, so their limits never interact.

Strengths: Keeps full sorted history per user, no destructive pruning; Correctly handles duplicates by using a list and strict inequality max-min < WINDOW; Handles out-of-order arrivals by inserting into sorted copy and checking all (LIMIT+1)-sized groups containing candidate; Passes the required test sequence; Per-user isolation via dict

Weaknesses: The comment claims 'any new violation must involve the candidate' but the loop doesn't explicitly restrict to groups containing the candidate — relies on invariant that prior history was valid, which is correct but subtle; Minor: window semantics defined as < 10 seconds span (inclusive of endpoints differing by 9) — matches standard interpretation but could be clarified; Unbounded memory growth since history is never pruned (acceptable per spec but worth noting)

The design is sound: by storing a sorted list of accepted timestamps and, for each new candidate, checking whether inserting it would create any 4-event group spanning < 10 seconds, it correctly enforces the rolling window under arbitrary arrival order. Duplicates are

preserved as separate list entries and the strict-less-than check treats same-timestamp events as co-located. The reliance on the invariant that stored history is always valid makes the loop-over-all-groups approach safe. Clarity is good and the explanation is within the 4-sentence limit. A minor nit is that memory grows unbounded, but the spec forbids destructive pruning, so this is appropriate.

Claude Sonnet 4.6

```python
class RateLimiter:
    """
    Rate limiter allowing at most 3 requests per user in any rolling
10-second window.

    Design requirements addressed:
    - Out-of-order timestamps: full history is retained (never pruned),
so a late-arriving
      timestamp can always be correctly evaluated against all prior
events.
    - Duplicate timestamps: each call appends independently to the
history list, so two
      calls at the same timestamp are counted as two distinct events.
    - Window-boundary correctness: the window for a request at time T
is the half-open
      interval (T-10, T], i.e. strictly more than 10 seconds ago are
excluded.
    """

    def __init__(self):
        # Maps user_id -> sorted list of accepted timestamps (with
duplicates)
        self._history: dict[int | str, list[int]] = {}

    # ------------------------------------------------------------------
-
    # Public API
    # ------------------------------------------------------------------
-

    def allow(self, user_id, timestamp: int) -> bool:
        """
        Decide whether the request (user_id, timestamp) should be
allowed.

        Parameters
        ----------
```

```
        user_id  : hashable identifier for the user
        timestamp: integer seconds

        Returns
        -------
        True  — request is within the rate limit and is recorded
        False — request would exceed the limit; it is NOT recorded
        """
        if user_id not in self._history:
            self._history[user_id] = []

        history = self._history[user_id]

        # Count how many *accepted* requests fall inside the rolling
window
        # Window for this request: (timestamp - 10, timestamp]
        # i.e. strictly within the last 10 seconds (inclusive of both
endpoints
        # when the gap is exactly 10 s the older event is *outside* the
window).
        window_start = timestamp - 10  # exclusive lower bound

        count = self._count_in_window(history, window_start, timestamp)

        if count < 3:
            # Accept: insert timestamp while keeping the list sorted so
that
            # bisect operations remain valid for future out-of-order
arrivals.
            self._sorted_insert(history, timestamp)
            return True

        return False

    # ----------------------------------------------------------------
-
    # Internal helpers
    # ----------------------------------------------------------------
-

    def _count_in_window(
        self, sorted_list: list[int], exclusive_start: int,
inclusive_end: int
    ) -> int:
        """
        Count elements x in sorted_list where exclusive_start < x <=
inclusive_end.
        Uses binary search for O(log n) boundary finding, O(1) count.
        """
```

```
        # Left boundary: first index where value > exclusive_start
        lo = self._bisect_right(sorted_list, exclusive_start)
        # Right boundary: first index where value > inclusive_end
        hi = self._bisect_right(sorted_list, inclusive_end)
        return hi - lo

    @staticmethod
    def _bisect_right(sorted_list: list[int], value: int) -> int:
        """
        Return the insertion point for *value* in *sorted_list* to keep
it sorted,
        with ties placed to the right (mirrors bisect.bisect_right).
        """
        lo, hi = 0, len(sorted_list)
        while lo < hi:
            mid = (lo + hi) // 2
            if sorted_list[mid] <= value:
                lo = mid + 1
            else:
                hi = mid
        return lo

    @staticmethod
    def _sorted_insert(sorted_list: list[int], value: int) -> None:
        """
        Insert *value* into *sorted_list* in O(n) while maintaining
sort order.
        (A bisect_right position is found in O(log n); the list shift
is O(n).)
        """
        lo, hi = 0, len(sorted_list)
        while lo < hi:
            mid = (lo + hi) // 2
            if sorted_list[mid] <= value:
                lo = mid + 1
            else:
                hi = mid
        sorted_list.insert(lo, value)
```

How the design handles out-of-order and duplicate timestamps

Out-of-order timestamps: The accepted-request history is **never pruned**; every past timestamp is kept indefinitely and stored in sorted order. When a late-arriving timestamp (e.g. `t=103` after `t=200` was already processed) is evaluated, a binary-search window count over the full, sorted history gives the exact number of accepted

```
requests in `(103-10, 103] = (93, 103]`, which correctly includes the
earlier `t=100`, `t=101`, and `t=102` events.

**Duplicate timestamps:** Each `allow` call is an independent event.
Accepted calls append one entry to the list regardless of whether an
identical timestamp already exists; two calls at `t=102` therefore
produce two entries and are counted separately, making the fourth call
(`allow(u, 102)`) correctly return `False`.

**Window boundary:** The window is defined as `exclusive_start < x <=
inclusive_end` (`x > timestamp - 10`), so a request exactly 10 seconds
earlier lies *outside* the window. This is why `allow(u, 200)` returns
`True`—all three prior requests (`t=100,101,102`) satisfy `x <= 200-10
= 190`, placing them outside the window.

**No pruning side-effects:** Because history is never deleted, a later
out-of-order decision can never lose a prior event that would have been
removed by premature pruning, satisfying the correctness requirement
for arbitrary arrival orders.
```

Strengths: Implements correct per-user rolling-window rate limiting with independent histories.; Correctly supports out-of-order timestamps by retaining full accepted history and keeping it sorted.; Correctly counts duplicate timestamps as separate events rather than collapsing them.; Uses boundary logic consistent with a 10-second rolling window: counts events where timestamp-10 < x <= timestamp.; Does not rely on destructive queue pruning, satisfying the non-monotonic stream requirement.; Given example sequence evaluates correctly, including allow(u, 200) → True and allow(u, 103) → False.

Weaknesses: Explanation exceeds the requested maximum of 4 sentences.; Type annotation for user_id is narrower than the docstring's stated 'hashable identifier' and may be unnecessarily specific.; Stores only accepted requests; while this is sufficient for the stated rules, the design choice is not explicitly justified against possible interpretations involving denied attempts.; Time complexity for insertion is O(n) per allowed request, which is acceptable here but not discussed as a tradeoff.

This is a high-quality implementation that directly addresses the benchmark's key pitfalls. The core logic is correct: it maintains per-user accepted timestamps in sorted order, uses binary search to count events in the half-open window (t-10, t], preserves duplicates, and never destructively prunes history, so later out-of-order requests can still be evaluated against earlier accepted events. The provided example sequence is handled correctly, including the non-monotonic case where a request at 103 arrives after one at 200. The main instruction-following flaw is that the post-code explanation is substantially longer than the allowed 4 sentences. Aside from that, the response is clear, complete, and aligned with the

prompt; the only minor technical nit is the somewhat restrictive user_id type annotation versus the more general stated interface.

OVH gpt-oss-120b

```
**Solution**

```python
import bisect
from collections import defaultdict

class RateLimiter:
    """
    Rate limiter that allows at most 3 requests per user in any rolling
    10-second window (inclusive).  Timestamps may arrive out of order
and
    many requests can share the same second.
    """

    def __init__(self):
        # For each user keep a *sorted* list of timestamps of the
        # requests that have been allowed.
        self.allowed = defaultdict(list)        # user_id -> [sorted
timestamps]

    def allow(self, user_id: str, timestamp: int) -> bool:
        """
        Return True if the request is allowed, otherwise False.
        """
        # Get the list for this user (sorted)
        history = self.allowed[user_id]

        # Determine the range of timestamps that fall into the
10-second window
        #   [timestamp-9, timestamp]  (both ends inclusive)
        left = timestamp - 9
        # left index — first element >= left
        lo = bisect.bisect_left(history, left)
        # right index — first element > timestamp
        hi = bisect.bisect_right(history, timestamp)

        # Number of already-allowed requests inside the window
        count_in_window = hi - lo

        if count_in_window >= 3:
            # Too many previous allowed requests → reject
            return False
```
```

```
        # Otherwise the request is allowed — insert it while keeping
the list sorted
        bisect.insort(history, timestamp)
        return True
```

Explanation (≤ 4 sentences)
The class stores, for each user, a sorted list of *allowed* timestamps; `bisect` lets us count in O(log n) how many of those lie in the inclusive window `[t-9, t]`, regardless of the order in which requests arrive. Because the list is never pruned, an out-of-order request can still see any earlier allowed timestamps that might affect its limit. Duplicate timestamps are stored separately, so they are each counted toward the three-request cap. Insertion via `bisect.insort` keeps the history sorted without relying on monotonic arrival order.

Strengths: Correctly handles out-of-order timestamps by maintaining a per-user sorted history and using binary search rather than append-only queue logic.; Correctly counts duplicate timestamps because each allowed request is inserted separately into the history list.; Uses an inclusive rolling window [timestamp-9, timestamp], which matches the provided example sequence and boundary expectations.; Tracks users independently with separate histories.; Does not destructively prune history, preserving information needed for later out-of-order decisions.; Explanation is concise and directly addresses out-of-order and duplicate timestamp handling.

Weaknesses: The implementation stores only allowed requests; while this is sufficient for enforcing the stated limit, it does not record rejected attempts, which could be seen as a narrower interpretation of 'history' though not incorrect here.; Time complexity is O(n) for insertion into Python lists, so scalability is not optimal for very large per-user histories, though correctness is unaffected.; The docstring says '10-second window (inclusive)', which is fine, but the exact boundary semantics could have been stated even more explicitly in relation to the sample behavior.

This response directly addresses the core difficulty of the prompt: requests can arrive out of chronological order, so queue-based pruning would be unsafe. By storing each user's allowed timestamps in sorted order and using bisect_left/bisect_right to count events in the inclusive interval [t-9, t], the implementation correctly handles boundary cases and duplicate timestamps. The sample sequence evaluates correctly: 100, 101, 102 are allowed; a second 102 is rejected because three prior allowed requests already fall in [93,102]; 200 is allowed; and then 103 is rejected because 100, 101, and 102 remain visible in [94,103]. The solution also respects the requirement not to mutate history in a way that breaks later out-of-order decisions, since it never prunes old events. Overall this is a highly accurate and well-

explained answer, with only minor efficiency limitations from list insertion and no meaningful correctness flaw for the stated requirements.

gpt-5.4

```python
class RateLimiter:
    def __init__(self):
        # user_id -> {timestamp: count}
        self.user_events = {}

    def allow(self, user_id, timestamp):
        if user_id not in self.user_events:
            self.user_events[user_id] = {}

        events = self.user_events[user_id]

        # Rolling 10-second window ending at `timestamp`:
        # [timestamp - 9, timestamp]
        window_start = timestamp - 9
        total_in_window = 0

        for ts, count in events.items():
            if window_start <= ts <= timestamp:
                total_in_window += count
                if total_in_window >= 3:
                    return False

        events[timestamp] = events.get(timestamp, 0) + 1
        return True
```

This stores per-user counts by exact timestamp, so duplicate timestamps are counted independently. For each decision, it scans all stored timestamps for that user and counts only those in the inclusive rolling window `[timestamp - 9, timestamp]`, so correctness does not depend on arrival order. Because it never destructively prunes history based on the latest seen timestamp, a later out-of-order request still has access to prior events it needs for a correct decision. The boundary is handled correctly: events exactly at `timestamp - 9` and `timestamp` are included.

Strengths: Correctly tracks users independently with separate per-user histories.; Handles out-of-order timestamps by evaluating against all stored events rather than relying on append order.; Correctly preserves duplicate timestamps via per-timestamp counts instead of collapsing them to a set.; Uses an inclusive rolling window [timestamp

- 9, timestamp], which matches the required 10-second window and boundary behavior.; Does not destructively prune history, so later out-of-order decisions still have access to prior events.; The provided required sequence evaluates correctly under this implementation.

Weaknesses: Time complexity is linear in the number of distinct stored timestamps per user for each request, so history can grow without bound.; The early return condition inside the scan is slightly subtle; while correct here, it depends on interpreting three prior requests in-window as making the current request the disallowed fourth.; No memory-management strategy is provided, though avoiding destructive pruning was a stated concern and correctness is preserved.

This implementation satisfies the prompt well. It keeps per-user state, counts duplicate timestamps correctly, and evaluates each request by scanning all stored timestamps for that user within the inclusive window [timestamp - 9, timestamp], so it remains correct even when requests arrive out of chronological order. It also avoids destructive pruning, which is important because pruning based on a latest-seen timestamp would break later out-of-order checks. The core logic is correct on the specified example and on window boundaries; the main limitation is efficiency and unbounded storage growth rather than correctness.

gpt-5.4-mini

```python
from bisect import bisect_left, bisect_right, insort


class RateLimiter:
    def __init__(self):
        # user_id -> sorted list of request timestamps
        self._history = {}

    def allow(self, user_id, timestamp):
        if not isinstance(timestamp, int):
            raise TypeError("timestamp must be an integer")

        history = self._history.setdefault(user_id, [])

        # Rolling 10-second window ending at timestamp:
        # include timestamps in [timestamp - 9, timestamp]
        start = timestamp - 9
        left = bisect_left(history, start)
        right = bisect_right(history, timestamp)
        count_in_window = right - left

        if count_in_window >= 3:
```

```
            return False

        # Keep all history so out-of-order later calls still see prior
events.
        insort(history, timestamp)
        return True
```

This keeps each user's timestamps in sorted order, so out-of-order arrivals are handled by counting only the requests in the exact window `[timestamp - 9, timestamp]` for each call. Duplicate timestamps are stored separately in the list, so each request is counted individually. The code does not prune old entries permanently, which avoids losing information needed for later out-of-order decisions. Window boundaries are handled correctly with `bisect_left` and `bisect_right`, including both endpoints of the 10-second window.

Strengths: Implements a per-user rate limiter with correct independent tracking via a user-to-history mapping.; Correctly handles out-of-order timestamps by keeping full sorted history rather than relying on append-only queue pruning.; Correctly counts duplicate timestamps because each accepted request is inserted as a separate list entry and bisect_right includes all equal timestamps.; Uses an inclusive rolling 10-second window [timestamp-9, timestamp], which matches the provided example and handles boundary cases correctly.; The provided sequence evaluates correctly: 100/101/102 allowed, second 102 denied, 200 allowed, 103 denied.; Explanation is concise and directly addresses out-of-order handling and duplicate timestamps within the 4-sentence limit.

Weaknesses: The implementation stores unbounded history forever, which is safe for correctness but may be memory-inefficient over long runtimes.; It raises TypeError for non-integer timestamps; this is reasonable but not explicitly required by the prompt.

This is a high-quality answer that squarely addresses the hard parts of the prompt. The design maintains a sorted timestamp list per user and uses bisect_left/bisect_right to count accepted requests within the inclusive window [timestamp - 9, timestamp], which is the right interpretation for a 10-second rolling window and matches the sample behavior. Because it never destructively prunes history, later out-of-order requests can still be evaluated against prior accepted events, satisfying the prompt's key constraint. Duplicate timestamps are handled correctly since each accepted request is inserted separately and equal timestamps are counted individually. The code is clear, uses only the standard library, and the explanation is accurate and within the sentence limit. The only notable downside is unbounded memory growth, but that is a performance tradeoff rather than a correctness flaw under this prompt.

gpt-5.4-nano

```python
class RateLimiter:
    def __init__(self):
        # user_id -> list of timestamps for accepted requests
        # We keep all timestamps and only remove those that are
strictly outside
        # the current window for that user's query.
        self.history = {}

    def allow(self, user_id, timestamp):
        if not isinstance(timestamp, int):
            raise TypeError("timestamp must be an integer (seconds)")

        win_start = timestamp - 9  # window is [timestamp-9,
timestamp], inclusive (10 seconds)

        times = self.history.get(user_id)
        if times is None:
            self.history[user_id] = [timestamp]
            return True

        # Prune only those timestamps that are definitely outside the
window.
        # Important: This pruning is safe even with out-of-order inputs
because it
        # removes only events older than the current window's left
boundary.
        # We may not know that the list is sorted, so we filter.
        times = [t for t in times if t >= win_start]
        # Store the pruned list back.
        self.history[user_id] = times

        # Count events in the inclusive window [win_start, timestamp]
        count = sum(1 for t in times if win_start <= t <= timestamp)

        # If fewer than 3 exist in the window, allow and record this
timestamp.
        if count < 3:
            times.append(timestamp)  # duplicates are appended and
count separately
            return True

        return False
```

This design handles out-of-order timestamps by keeping a per-user

```
history and, on each call, pruning only events strictly older than the
current window start (`timestamp - 9`), which cannot affect future
decisions for later timestamps. It counts how many stored events fall
within the inclusive window `[timestamp-9, timestamp]` regardless of
insertion order. Duplicate timestamps are stored as separate entries
(each `append` adds one), so they are counted separately. Window-
boundary correctness is ensured by using inclusive comparisons (`t >=
win_start` and `t <= timestamp`).
```

Strengths: Implements a `RateLimiter` class with the required `allow(user_id, timestamp)` method.; Tracks users independently via a per-user history map.; Counts duplicate timestamps separately by storing each accepted request as a separate list entry.; Uses an inclusive 10-second window interpretation `[timestamp-9, timestamp]`, which matches the boundary example.

Weaknesses: Core logic is incorrect for out-of-order requests because it destructively prunes history based on the current request timestamp. After handling a later timestamp like 200, it removes earlier accepted events such as 100/101/102, causing a later out-of-order call at 103 to be misjudged.; The explanation incorrectly claims pruning is safe for out-of-order inputs and even says removed events cannot affect future decisions for later timestamps, but the prompt explicitly requires correctness for later out-of-order decisions as well.; Violates the requirement not to mutate stored history in a way that loses needed prior events.; Fails the required test sequence: after `allow(u,200) → True`, the implementation would return `True` for `allow(u,103)` instead of the required `False`.; Relies on pruning logic that is only safe for monotonic or partially monotonic streams, despite the prompt explicitly disallowing queue-only/pruning assumptions.

The candidate addresses several surface requirements: it defines the requested class and method, tracks users independently, and preserves duplicate accepted requests as separate list entries. It also uses inclusive boundary checks consistent with a rolling 10-second window interpreted as `[timestamp-9, timestamp]`. However, the central requirement of supporting out-of-order timestamps is not met. The implementation overwrites each user's history with only timestamps `>= win_start` for the current request, which permanently discards older accepted events. That is unsafe when a later call arrives with an earlier timestamp. In the required sequence, processing timestamp 200 prunes away 100, 101, and 102; then a subsequent request at 103 sees no prior events in `[94,103]` and is incorrectly allowed, whereas it should be denied because 100, 101, and 102 were previously accepted and still fall in that window. This directly violates both correctness and the explicit instruction not to mutate stored history in a way that loses needed prior events. The explanatory paragraph reinforces the mistake by asserting the pruning is safe with out-of-order inputs, which is false. So while the code is readable and partially aligned with the API, it fails the benchmark's key behavioral requirement.

write-customer-response-with-hidden-conflict-constraints

Writing

SYSTEM PROMPT

```
Write with empathy and professionalism. Respect every explicit constraint
exactly, especially the ones that conflict in subtle ways.
```

USER PROMPT

```
Write a reply to a customer whose order arrived 5 days late and who is
asking for a full refund.

Constraints:
- Between 95 and 105 words inclusive
- Exactly 2 paragraphs
- Acknowledge frustration
- Apologize clearly
- Offer a 15% discount on the next order
- Do not offer a refund
- Do not blame the shipping carrier
- Do not use the words `policy`, `refund denied`, `unfortunately`, `cannot`,
or `won't`
- Include exactly one sentence that begins with `As a gesture of goodwill,`
- The second paragraph must contain exactly 2 sentences
- The final sentence must invite the customer to reply directly
```

Evaluation criteria: Reward strict compliance with length, paragraph structure, forbidden phrase constraints, and the required goodwill sentence while still sounding natural and empathetic. Penalize accidental carrier blame, blunt refusal language, or implicit refund promises.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**55.00**	#1	4700 ms	481	$0.0189750000	11.43
Claude Sonnet 4.6	**90.00**	#1	3345 ms	314	$0E-10	28.66
OVH gpt-oss-120b	**55.00**	#1	26407 ms	1992	$0.0007212800	2.76
gpt-5.4	**82.00**	#1	2675 ms	293	$0.0022325000	27.99

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
gpt-5.4-mini	**62.00**	#1	1662 ms	283	$0.0006247500	21.91
gpt-5.4-nano	**62.00**	#1	2138 ms	290	$0.0001808500	21.38

Claude Opus 4.7

```
Dear Customer,

Thank you for reaching out, and I completely understand how frustrating
it is to wait five extra days for an order you were counting on. Please
accept my sincere apology for the delay and the inconvenience it caused
you. We take full ownership of this experience and are actively
reviewing our fulfillment process to ensure deliveries reach you on
time going forward. Your patience during this situation means a great
deal to us, and your feedback helps us improve.

As a gesture of goodwill, I would like to offer you a 15% discount on
your next order with us. Please reply directly to this message so I can
share your discount code.
```

Strengths: Acknowledges frustration and apologizes clearly; Includes required 'As a gesture of goodwill,' sentence; Avoids forbidden words and does not blame carrier; Final sentence invites direct reply; Offers 15% discount without promising refund

Weaknesses: Word count is roughly 113-115 words, exceeding the 95-105 range; Contains three paragraphs if the salutation 'Dear Customer,' counts, or arguably two body paragraphs; structure is ambiguous; Second paragraph has 2 sentences (compliant) but overall length violates constraint

The reply handles tone well: it acknowledges frustration, apologizes sincerely, avoids all forbidden terms, sidesteps carrier blame, includes the mandated 'As a gesture of goodwill,' sentence, and closes with an invitation to reply directly. The second paragraph correctly contains exactly two sentences. However, the body runs about 113-115 words, clearly outside the 95-105 inclusive range, which is a hard failure. Additionally, 'Dear Customer,' as a standalone salutation line could be read as a third paragraph, creating structural ambiguity against the 'exactly 2 paragraphs' rule. These compliance misses significantly reduce the score despite otherwise natural, empathetic phrasing.

Claude Sonnet 4.6

```
Thank you for reaching out, and we sincerely apologize for the delay in
receiving your order. We completely understand how frustrating it is to
wait longer than expected, and we take full responsibility for the
experience falling short of your expectations. We know your time
matters, and a five-day delay is simply not acceptable.

As a gesture of goodwill, we'd like to offer you 15% off your next
order with us. We hope this small acknowledgment helps rebuild your
confidence in us, and please reply directly to this message so we can
assist you further.
```

Strengths: Meets the core customer-service goal with a polite, empathetic reply.; Clearly acknowledges frustration and apologizes.; Includes exactly one sentence beginning with "As a gesture of goodwill," and offers the required 15% discount on the next order.; Uses exactly 2 paragraphs, with the second paragraph containing exactly 2 sentences.; Avoids the forbidden words and does not blame the shipping carrier.; Ends with an invitation for the customer to reply directly.

Weaknesses: Word count is 92, which misses the required 95 to 105 word range.; Does not explicitly address the customer's request for a full refund by declining it; while it avoids offering a refund, the omission makes the response slightly less complete for the scenario.

The response performs well on tone, empathy, and most structural constraints. It clearly apologizes, acknowledges the customer's frustration, takes responsibility, and offers the required 15% discount in a sentence that correctly begins with "As a gesture of goodwill,". It also maintains exactly two paragraphs, and the second paragraph has exactly two sentences. The final sentence appropriately invites the customer to reply directly. It avoids all prohibited terms and does not shift blame to the shipping carrier. The main compliance issue is length: the response is 92 words, below the required 95 to 105 range. That is a significant instruction-following miss because the benchmark explicitly rewards strict compliance. Additionally, although the prompt is about a customer asking for a full refund, the reply does not directly address that request; it avoids offering a refund, which is correct, but it leaves the refusal implicit rather than tactfully stated. Overall, this is a strong customer-service message with one clear hard-constraint failure and one minor completeness gap.

OVH gpt-oss-120b

```
Thank you for reaching out and sharing experience. We understand how
frustrating it is when an order arrives later than expected, and we
sincerely apologize for the delay. Our team strives to ensure timely
deliveries, and we are sorry this time fell short of that standard. We
appreciate your patience and want to assure you that we are reviewing
our processes to prevent similar issues in the future.

As a gesture of goodwill, we would like to provide a 15% discount on
your next order. Please let us know how else we may assist you, and
feel free to reply directly to this message.
```

Strengths: Acknowledges frustration and apologizes clearly; Includes the required 'As a gesture of goodwill,' sentence; Avoids forbidden words and does not blame the carrier; Invites the customer to reply directly at the end

Weaknesses: Word count is approximately 108 words, exceeding the 95-105 range; Second paragraph contains 3 sentences, not the required 2; Does not explicitly decline the refund request, leaving ambiguity; Minor grammatical issue ('sharing experience')

The reply is empathetic and hits most tonal requirements, including the goodwill sentence, 15% discount offer, apology, and invitation to reply. However, strict compliance fails on two key structural constraints: the total word count exceeds 105, and the second paragraph contains three sentences rather than exactly two. Additionally, while the response avoids forbidden refusal words, it never addresses the refund request itself, which could leave the customer expecting one. A minor grammar slip ('sharing experience' missing 'your') slightly reduces polish.

gpt-5.4

```
I understand how frustrating it is to receive your order five days
later than expected, especially when you were counting on it arriving
sooner. I'm very sorry for the delay and for the inconvenience and
disappointment this caused. Your experience matters to us, and we
appreciate you bringing this to our attention so we can review what
happened and work to improve future deliveries.

As a gesture of goodwill, we'd like to offer you a 15% discount on your
next order. Please reply directly if you'd like us to apply that for
you or if there's anything else we can do to help.
```

Strengths: Acknowledges frustration clearly; Clear apology without forbidden words; Includes the exact 'As a gesture of goodwill,' sentence; Two paragraphs with second

paragraph containing exactly two sentences; Final sentence invites direct reply; Avoids carrier blame and refund promises

Weaknesses: Word count is approximately 108-110 words, exceeding the 95-105 limit; Does not explicitly address the refund request, which could feel evasive

The reply hits most constraints: two paragraphs, the second with exactly two sentences, the mandated 'As a gesture of goodwill,' sentence, an empathetic acknowledgment, a clear apology, the 15% discount offer, no forbidden terms, no carrier blame, and a closing invitation to reply. However, counting the words yields roughly 108, which breaches the strict 95-105 word window—a hard constraint. It also sidesteps the refund request entirely rather than gently declining, which is acceptable but slightly less responsive. Tone is warm and professional.

gpt-5.4-mini

```
I'm sorry your order arrived 5 days late, and I understand how
frustrating that must have been. We know timeliness matters, and I
apologize for the delay and any inconvenience it caused. While I'm not
able to provide a full refund, I'd like to help make this right with
15% off your next order.

As a gesture of goodwill, I've noted the discount for your account and
hope it helps on a future purchase. If you'd like, please reply
directly and I'll be happy to assist further.
```

Strengths: Acknowledges frustration and apologizes clearly; Avoids forbidden words; Does not blame carrier; Offers 15% discount; Includes 'As a gesture of goodwill,' sentence; Final sentence invites direct reply; Two paragraphs with second paragraph containing exactly 2 sentences

Weaknesses: Word count is approximately 88 words, below the 95-105 requirement; Phrase 'not able to provide a full refund' comes close to blunt refusal language

The reply meets most constraints: two paragraphs, second paragraph has exactly two sentences, forbidden words are avoided, carrier is not blamed, the goodwill sentence is present, and the final sentence invites a direct reply. However, the total word count is around 88 words, which violates the 95-105 inclusive requirement—a hard constraint failure. The refusal phrasing ('I'm not able to provide a full refund') is acceptable but borderline blunt. Tone is empathetic and natural, but the length miss is a significant instruction-following penalty.

gpt-5.4-nano

```
We're sorry your order arrived 5 days late, and we understand how
frustrating that is. We apologize clearly for the delay and for any
inconvenience it caused your plans. As a gesture of goodwill, we'd like
to offer you 15% off your next order. Please use the code DELAY15 at
checkout.

Thank you for your patience, and we're committed to making this right
in a practical way. If you share your order number and preferred items,
we'll ensure the discount is applied properly and help with any next
steps you need—please reply directly.
```

Strengths: Acknowledges frustration and apologizes; Includes the required 'As a gesture of goodwill,' sentence; Offers 15% discount without promising refund; Avoids forbidden words and carrier blame; Two paragraphs with second paragraph having 2 sentences

Weaknesses: Word count appears to be around 95-97 words - borderline but likely compliant; needs exact check; Final sentence invites reply only as an appended clause ('please reply directly') rather than a clear invitation starting the sentence; The final sentence is compound and the 'reply directly' feels tacked on; First paragraph has 4 sentences which is fine, but overall flow is slightly mechanical ('We apologize clearly')

The response hits the key structural requirements: two paragraphs, second paragraph with exactly two sentences, the mandated 'As a gesture of goodwill,' sentence, a 15% discount offer, no refund promise, no carrier blame, and no forbidden words. Word count lands near the lower bound (~96 words), likely within 95-105. Weaknesses: the phrase 'We apologize clearly' reads as meta/robotic rather than a natural apology. The final sentence does invite a reply but buries it after an em-dash, which weakens the directness of the invitation. Tone is adequate but not especially warm.

reasoning-multi-rule-approval-matrix

Reasoning

SYSTEM PROMPT

```
Apply the rules exactly as written. Respect precedence and scope. Do not
extend exceptions beyond what is explicitly stated.
```

USER PROMPT

```
A company applies these approval rules:
1. No one may approve their own request.
2. Finance-approved managers may approve any expense up to 5000 euros.
3. Managers who are not finance-approved may approve non-travel expenses
below 1000 euros.
4. Temporary managers may not approve travel expenses.
5. A person on the exception list for a specific expense type may approve
that type below 1000 euros unless blocked by a higher-priority rule.
6. However, the temporary-manager travel restriction in rule 4 does not
apply to exception-list approvals below 500 euros.

Facts:
- Elena is a manager.
- Elena is not finance-approved.
- Elena is temporary.
- Elena is on the exception list for travel.
- Request A is a 300 euro travel expense submitted by Marco.
- Request B is a 700 euro software expense submitted by Priya.
- Request C is a 700 euro travel expense submitted by Elena.
- Request D is a 1200 euro travel expense submitted by Omar.
- Request E is a 450 euro travel expense submitted by Omar.

Answer in exactly this format:
`A: yes/no - reason`
`B: yes/no - reason`
`C: yes/no - reason`
`D: yes/no - reason`
`E: yes/no - reason`
```

Evaluation criteria: Reward exact precedence and scope handling: A yes because rule 6 creates a narrow carve-out, B yes via rule 3, C no due to self-approval, D no because the exception does not extend above 1000, and E yes via the same carve-out. Penalize treating rule 5 as broader than written.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**72.00**	#1	11866 ms	1102	$0.0536100000	6.53
Claude Sonnet 4.6	**72.00**	#1	12371 ms	1144	$0E-10	6.29
OVH gpt-oss-120b	**98.00**	#1	9299 ms	1511	$0.0004885600	6.49
gpt-5.4	**78.00**	#1	2668 ms	461	$0.0031775000	16.92
gpt-5.4-mini	**99.00**	#1	1555 ms	457	$0.0009352500	21.66
gpt-5.4-nano	**54.00**	#1	2216 ms	496	$0.0003060500	10.89

Claude Opus 4.7

```
A: yes - travel exception list below 1000; rule 6 exempts rule 4 since
under 500... wait, 300 is below 500, so rule 4 doesn't block; approved
B: no - not finance-approved, and software at 700 is not below 1000? It
is below 1000 but she's not on exception list for software; rule 3
allows non-travel below 1000, so yes

Let me redo this properly:

A: yes - non-finance manager, but on travel exception list; 300 < 1000
(rule 5), and rule 6 exempts the rule 4 travel restriction since 300 <
500
B: yes - rule 3 allows non-finance managers to approve non-travel
expenses below 1000 euros (700 < 1000)
C: no - rule 1 forbids approving one's own request
D: no - 1200 exceeds the 1000 euro exception-list limit, and she is not
finance-approved; also temporary manager cannot approve travel (rule 4)
E: no - 450 is below 500 so rule 6 exempts rule 4, and she is on travel
exception list below 1000, so rule 5 permits — yes

Final answer:

`A: yes - Elena is on the travel exception list and 300 < 1000 (rule
5); rule 6 waives the rule 4 travel restriction since 300 < 500`
`B: yes - rule 3 permits a non-finance manager to approve a non-travel
```

```
expense below 1000 euros (700 < 1000)`
`C: no - rule 1 prohibits approving one's own request`
`D: no - 1200 exceeds the 1000 euro exception-list cap and her 5000
finance limit does not apply; rule 4 also blocks travel for temporary
managers`
`E: yes - Elena is on the travel exception list and 450 < 1000 (rule
5); rule 6 waives the rule 4 restriction since 450 < 500`
```

Strengths: Final answer includes all five requested items A through E.; Correctly applies the key rules for A, B, C, and D, including the narrow carve-out in rule 6 for A.; Shows understanding that self-approval overrides other possible permissions for C.; Uses the required yes/no with reason structure in the final section.

Weaknesses: The response contains extraneous draft reasoning and self-corrections before the final answer, so it does not strictly answer in exactly the requested format.; E is internally inconsistent: the final line says 'yes', but the reasoning depends on 450 < 500; while that inequality is true here, earlier draft text says 'E: no' before correcting, reducing reliability.; The rationale for D could be cleaner on precedence: rule 4 blocks temporary-manager travel, and the exception-list route fails because 1200 exceeds the rule 5 limit; mentioning a '5000 finance limit' is unnecessary since Elena is not finance-approved.; Because the benchmark explicitly requests exact precedence and scope handling, the messy intermediate contradictions detract from precision.

On the core logic, the candidate gets the substantive outcomes right in the final answer: A yes via the travel exception list with the rule 6 carve-out below 500; B yes via rule 3 for a non-travel expense below 1000; C no due to self-approval; D no because the exception-list approval does not extend to 1200 and Elena is also blocked as a temporary manager approving travel; E yes via the same carve-out as A. That matches the intended evaluation notes. However, the answer does not comply with the instruction to answer in exactly the specified format because it includes preliminary false starts, corrections, and a 'Final answer' wrapper before the formatted lines. This should be penalized on instruction following. The intermediate reasoning also includes contradictions, especially around B and E, which undermines clarity and confidence even though the final formatted answers are mostly correct. Overall, this is a good substantive answer with notable formatting and presentation defects.

Claude Sonnet 4.6

```
I'll analyze each request for whether Elena can approve it.

**Setting up Elena's profile:**
- Manager ✓
- Not finance-approved
```

- Temporary
- On exception list for travel

Request A: 300 euro travel expense, submitted by Marco

Elena is temporary → Rule 4 blocks travel approvals.
But Elena is on the exception list for travel → Rule 5 allows travel below 1000 euros.
Rule 6 states the temporary-manager travel restriction (Rule 4) does NOT apply to exception-list approvals below 500 euros.
300 < 500, so Rule 6 lifts Rule 4's restriction.
Rule 1: Elena ≠ Marco ✓
→ Elena CAN approve.

`A: yes - Elena is on the travel exception list, the amount is below 500 euros so Rule 6 lifts the temporary-manager travel restriction, and she is not approving her own request`

Request B: 700 euro software expense, submitted by Priya

Software is a non-travel expense. Elena is not finance-approved.
Rule 3: Non-finance-approved managers may approve non-travel expenses below 1000 euros. 700 < 1000 ✓
Rule 1: Elena ≠ Priya ✓
Rule 4 does not apply (not travel).
→ Elena CAN approve.

`B: yes - software is a non-travel expense below 1000 euros, so Rule 3 permits Elena (a non-finance-approved manager) to approve it, and she is not approving her own request`

Request C: 700 euro travel expense, submitted by Elena

Rule 1: Elena cannot approve her own request. This is a higher-priority block.
→ Elena CANNOT approve.

`C: no - Rule 1 prohibits Elena from approving her own request`

Request D: 1200 euro travel expense, submitted by Omar

```
Elena is on the travel exception list → Rule 5 allows travel below 1000
euros. 1200 ≥ 1000, so Rule 5 does not apply.
Rule 4 blocks travel for temporary managers.
Rule 6 only lifts Rule 4 for exception-list approvals below 500 euros;
1200 ≥ 500, so Rule 6 does not help.
Elena is not finance-approved, so Rule 2 does not apply.
→ Elena CANNOT approve.

`D: no - 1200 euros exceeds the 1000 euro exception-list threshold in
Rule 5, Rule 4 blocks temporary managers from travel approvals, and
Rule 6 does not lift that restriction above 500 euros`

---

**Request E: 450 euro travel expense, submitted by Omar**

Elena is on the travel exception list → Rule 5 would allow travel below
1000 euros, but Rule 4 blocks temporary managers on travel.
Rule 6: The temporary-manager travel restriction does NOT apply to
exception-list approvals below 500 euros. 450 < 500 ✓
Rule 1: Elena ≠ Omar ✓
→ Elena CAN approve.

`E: yes - Elena is on the travel exception list, the amount is below
500 euros so Rule 6 lifts the temporary-manager travel restriction, and
she is not approving her own request`
```

Strengths: Reasoning about rule precedence and carve-out in rules 5 and 6 is correct.; All five request outcomes are correct: A yes, B yes, C no, D no, E yes.; Explanations clearly cite the applicable rules and amounts.

Weaknesses: Did not follow the required output format exactly; included substantial extra analysis and headings instead of only the five specified lines.; Response is not in exactly the mandated final format, despite embedding correctly formatted lines within a longer answer.

The candidate gets the logic right across all cases. It correctly handles the narrow carve-out in rule 6 for temporary managers on the exception list approving travel below 500 euros, leading to A=yes and E=yes. It correctly applies rule 3 for B=yes, rule 1 for C=no, and recognizes that the exception-list authority does not extend to D because 1200 exceeds the rule 5 threshold and rule 6 does not apply. This matches the evaluation notes exactly. However, the benchmark explicitly requires answering in exactly the five-line format shown. The candidate adds setup, headings, bullets, and explanatory prose outside that format. Since the prompt makes format compliance explicit, this is a significant instruction-following failure even though the embedded verdict lines themselves are correct.

OVH gpt-oss-120b

```
A: yes - Elena is on the exception list for travel and the amount
(300 €) is below 500 €, so rule 6 lifts the temporary-manager travel
restriction.
B: yes - As a non-finance-approved manager, Elena may approve
non-travel expenses under 1000 € per rule 3.
C: no - Rule 1 prohibits anyone from approving their own request; Elena
submitted request C.
D: no - Elena is a temporary manager (rule 4) and the travel expense
exceeds 1000 €, so neither rule 5 nor rule 6 provide an exception.
E: yes - The travel expense is 450 €, below 500 €, so rule 6 overrides
the temporary-manager travel restriction, allowing Elena to approve it.
```

Strengths: Correctly applies the self-approval prohibition to Request C.; Correctly uses rule 3 for Request B.; Correctly handles the narrow carve-out in rule 6 for Requests A and E.; Correctly rejects Request D because the exception logic does not extend to that amount/context.; Follows the requested output structure exactly.

Weaknesses: The reasoning for A and E cites rule 6 lifting the temporary-manager restriction but does not explicitly mention that approval still comes via exception-list authority under rule 5.; The explanation for D is slightly compressed and could more explicitly state that rule 5 only applies below 1000 euros.

The candidate response is essentially fully correct. It gets the intended outcomes exactly right: A yes, B yes, C no, D no, E yes. It properly treats rule 6 as a narrow carve-out to rule 4 rather than as a standalone broad approval rule, which is crucial for A and E. It also correctly applies rule 3 to B and rule 1 to C. For D, it correctly denies approval because the request is travel, Elena is temporary, and the exception-list pathway does not apply at 1200 euros; therefore neither rule 5 nor rule 6 can authorize it. The only minor limitation is that for A and E the explanation could have been more explicit that rule 5 supplies the underlying approval authority for travel below 1000 euros, while rule 6 merely removes the temporary-manager travel bar below 500 euros. This is a subtle presentation issue rather than a substantive mistake. Overall, the answer demonstrates excellent accuracy, completeness, and instruction adherence.

gpt-5.4

```
A: yes - Elena is on the exception list for travel, the amount is below
1000 euros, and the temporary-manager travel restriction does not apply
to exception-list approvals below 500 euros.
B: yes - Elena is a manager who is not finance-approved, and may
approve non-travel expenses below 1000 euros.
```

```
C: no - no one may approve their own request.
D: no - Elena is not finance-approved, rule 3 does not apply because it
is travel, and the exception-list rule does not apply because the
amount is not below 1000 euros.
E: yes - Elena is on the exception list for travel, the amount is below
1000 euros, and the temporary-manager travel restriction does not apply
to exception-list approvals below 500 euros.
```

Strengths: Covers all five requests in the required line-by-line format.; Correctly applies the self-approval prohibition to C.; Correctly approves B under the non-finance-approved manager rule for non-travel expenses below 1000 euros.; Correctly uses the carve-out in rule 6 to approve A and E.; Clear, concise reasoning tied to the stated rules.

Weaknesses: The reason for D is incorrect: 1200 euros is in fact not below 1000, but the stated explanation says the exception-list rule does not apply because the amount is not below 1000 euros, while 1200 is indeed not below 1000; however, the response misses the more precise precedence/scope explanation that the exception rule only covers below 1000 and the carve-out only below 500.; The evaluation note expects careful handling of rule scope and precedence; the candidate's D explanation is somewhat under-specified.; A and E are correctly answered, but the reasoning could more explicitly note that rule 5 would normally be blocked by rule 4 and that rule 6 narrowly restores approval only below 500 euros.

This candidate response is strong overall. It follows the exact required output structure, addresses every request, and gets the key approvals right: A yes via the exception-list travel approval plus the narrow carve-out under 500 euros, B yes via the manager/non-finance/non-travel/below-1000 rule, C no due to self-approval, and E yes via the same carve-out as A. These are the core reasoning points the prompt emphasizes. The main issue is with D's explanation. The final answer 'no' is correct, but the reasoning is only partially precise. The candidate says the exception-list rule does not apply because the amount is not below 1000 euros; that is logically correct for 1200 euros, but the response does not fully articulate the intended scope handling that rule 5 is limited to below 1000 and rule 6 is an even narrower carve-out below 500 from the temporary travel restriction. So while the conclusion is correct, the explanation is thinner than ideal for a benchmark centered on precedence and scope. Still, because the answers themselves are correct and the formatting is exact, this is a high-quality response, just not perfect on explanatory precision.

gpt-5.4-mini

```
A: yes - Elena is on the travel exception list and the expense is below
1000 euros; it is also below 500 euros, so the temporary-manager travel
restriction does not apply.
B: yes - Elena is a manager who is not finance-approved, so she may
```

```
approve non-travel expenses below 1000 euros.
C: no - Elena cannot approve her own request.
D: no - the travel expense is above 1000 euros, so the exception-list
rule does not apply, and temporary managers may not approve travel
expenses.
E: yes - Elena is on the travel exception list and the travel expense
is below 1000 euros; since it is below 500 euros, the temporary-manager
travel restriction does not apply.
```

Strengths: Correctly applies rule precedence and exceptions across all five requests.; Identifies the narrow carve-out in rule 6 for temporary managers approving travel below 500 euros.; Correctly denies self-approval for Request C.; Uses the exact requested output format with one line per request.; Reasons are concise and directly tied to the governing rule in each case.

Weaknesses: For Request A, the justification could more explicitly note that Elena is not the submitter, though the conclusion is still correct.; For Request D, the explanation mentions the temporary-manager travel restriction after noting rule 5 does not apply; this is correct but could be slightly more explicit about why no other rule authorizes approval.

The candidate response is essentially fully correct. A is correctly marked yes because Elena is on the travel exception list, the amount is below 1000 euros under rule 5, and rule 6 removes the temporary-manager travel restriction for exception-list approvals below 500 euros. B is correctly marked yes under rule 3 because Elena is a manager, not finance-approved, and the software expense is non-travel and below 1000 euros. C is correctly marked no due to the highest-priority self-approval prohibition in rule 1. D is correctly marked no because the travel exception in rule 5 only applies below 1000 euros, and as a temporary manager Elena is otherwise barred from approving travel under rule 4. E is correctly marked yes for the same carve-out logic as A: exception-list travel approval below 1000 euros, with rule 6 overriding rule 4 below 500 euros. The response also follows the exact requested formatting and gives concise reasons. Any imperfections are minimal and stylistic rather than substantive.

gpt-5.4-nano

```
A: yes - Elena is exception-list for travel and 300€ is below 500€;
rule 4's temporary travel restriction does not apply to exception-list
approvals below 500€
B: no - Elena is not finance-approved and 700€ is above 1000€? (it is
below 1000, but Elena cannot approve travel exception restriction
doesn't cover software; rule 5 applies only to the exception-list
expense type, which is travel)
C: no - rule 4 blocks temporary managers from approving travel
```

```
expenses, and rule 6 only relaxes this for exception-list travel
approvals below 500€; 700€ is not below 500€
D: no - Elena is temporary and rule 4 blocks approval of travel
expenses for temporary managers
E: yes - Elena is exception-list for travel and 450€ is below 500€;
rule 4's temporary travel restriction does not apply to exception-list
approvals below 500€
```

Strengths: Correctly identifies A as approvable via the narrow rule 6 carve-out for exception-list travel approvals below 500 euros.; Correctly identifies D as not approvable.; Correctly identifies E as approvable via the same below-500-euro carve-out.; Generally engages with the rule-precedence issue around temporary-manager travel restrictions and the exception-list carve-out.

Weaknesses: Gets B wrong: Elena can approve B under rule 3 because she is a manager, not finance-approved, and B is a non-travel expense below 1000 euros.; Gets C wrong for the wrong reason and misses the highest-priority self-approval prohibition; C should be denied because Elena submitted it herself.; Reasoning for B is confused and internally contradictory, explicitly noting 700 euros is below 1000 while still concluding no.; Does not fully reflect the required precedence handling, especially that self-approval overrides everything and that non-travel approval under rule 3 is independent of travel exception status.; Does not strictly follow the requested exact output format because the reasons are more elaborate than necessary and include parenthetical commentary, though the line structure is close.

The response gets A, D, and E correct. A and E are correctly approved because Elena is on the exception list for travel and rule 6 creates a narrow carve-out: the temporary-manager travel restriction in rule 4 does not apply to exception-list approvals below 500 euros. D is correctly denied because 1200 euros exceeds the scope of the exception-list rule and Elena is also barred by the temporary-manager travel restriction. However, B is incorrect. Request B is a 700 euro software expense, which is non-travel and below 1000 euros. Since Elena is a manager and not finance-approved, rule 3 allows her to approve non-travel expenses below 1000 euros. Her exception-list status for travel is irrelevant here. The candidate's reasoning is contradictory and shows confusion about the thresholds and scope. C is also mishandled. The final yes/no answer of 'no' matches the correct outcome, but the reasoning fails to apply the highest-priority and most direct rule: no one may approve their own request. Because Elena submitted request C herself, that alone is sufficient to deny approval. The answer instead focuses on rule 4 and rule 6, missing the critical self-approval prohibition emphasized in the evaluation notes. Overall, the candidate demonstrates some understanding of the travel-exception carve-out but makes important errors in rule scope and precedence.

transform-contract-clauses-to-strict-json

Structured Output

SYSTEM PROMPT

```
Return valid JSON only. Follow the schema exactly and obey the inclusion and
exclusion rules without approximation.
```

USER PROMPT

```
Convert the clauses below into a JSON array. Each object must contain
exactly these keys:
`clause_id`, `owner`, `action`, `deadline_days`, `applies_if`, `excluded_if`

Rules:
- `owner` must be one of: `vendor`, `client`, `both`
- `deadline_days` must be an integer or null
- `applies_if` and `excluded_if` must be strings or null
- Include only clauses that create an affirmative obligation
- Exclude clauses that are purely definitional, purely permissive, or state
only a consequence without an action
- If a clause contains both an obligation and an exception, keep both in the
same object
- If a clause contains a temporal or triggering condition, put it in
`applies_if` rather than `action`
- Keep exceptions and blockers in `excluded_if`, even if they appear after
the main obligation
- Return JSON only

Clauses:
1. C1: The Vendor shall notify the Client of any security incident within 3
days of confirmation unless law enforcement prohibits disclosure.
2. C2: For the purposes of this Agreement, "Confidential Information"
includes customer lists and pricing annexes.
3. C3: The Client may request an audit once per calendar year.
4. C4: Each party shall maintain commercially reasonable backups during the
Term.
5. C5: If the Vendor fails to meet the service levels for 3 consecutive
months, service credits will apply.
6. C6: The Client must provide relevant access credentials within 5 days
after written onboarding notice, except where prohibited by internal policy.
7. C7: Either party may terminate the Agreement for convenience with 30
days' notice.
8. C8: During any approved disaster-recovery test, the Vendor must restore
the staging environment within 2 days unless the Client has not delivered
the required seed data.
9. C9: The Vendor may archive support logs after 90 days.
```

```
10. C10: Upon written request by the Client, each party shall designate a
security contact within 7 days, except while a merger-related
confidentiality hold is active.
```

Evaluation criteria: Reward inclusion of C1, C4, C6, C8, and C10 only; correct owner normalization; proper extraction of deadlines, trigger conditions, and exceptions; and exclusion of definitional, permissive, and consequence-only clauses.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**92.00**	#1	5269 ms	1127	$0.0414450000	8.16
Claude Sonnet 4.6	**92.00**	#1	4596 ms	924	$0E-10	9.96
OVH gpt-oss-120b	**96.00**	#1	16680 ms	1929	$0.0006029600	4.98
gpt-5.4	**97.00**	#1	4060 ms	794	$0.0061100000	12.22
gpt-5.4-mini	**99.00**	#1	2475 ms	790	$0.0018150000	12.53
gpt-5.4-nano	**88.00**	#1	5133 ms	799	$0.0005115500	11.01

Claude Opus 4.7


```
[
{"clause_id": "C1", "owner": "vendor", "action": "notify the Client of
any security incident", "deadline_days": 3, "applies_if": "within 3
days of confirmation of a security incident", "excluded_if": "law
enforcement prohibits disclosure"},
{"clause_id": "C4", "owner": "both", "action": "maintain commercially
reasonable backups", "deadline_days": null, "applies_if": "during the
Term", "excluded_if": null},
{"clause_id": "C6", "owner": "client", "action": "provide relevant
access credentials", "deadline_days": 5, "applies_if": "after written
onboarding notice", "excluded_if": "prohibited by internal policy"},
{"clause_id": "C8", "owner": "vendor", "action": "restore the staging
environment", "deadline_days": 2, "applies_if": "during any approved
disaster-recovery test", "excluded_if": "the Client has not delivered
the required seed data"},
```

```
{"clause_id": "C10", "owner": "both", "action": "designate a security
contact", "deadline_days": 7, "applies_if": "upon written request by
the Client", "excluded_if": "a merger-related confidentiality hold is
active"}
]
```

Strengths: Correctly includes exactly C1, C4, C6, C8, C10; Proper owner normalization including 'both' for each-party clauses; Accurate extraction of deadlines, triggers, and exceptions; Valid JSON with exact key schema

Weaknesses: C1 applies_if redundantly repeats the 3-day deadline already in deadline_days; trigger should be just 'confirmation of a security incident'; Minor: could have put 'during the Term' more cleanly but acceptable

The candidate correctly filters out definitional (C2), permissive (C3, C7, C9), and consequence-only (C5) clauses. Owners are normalized properly, with 'both' used for C4 and C10. Deadlines are integers where present and null for C4. Exceptions are placed in excluded_if as required. The only notable issue is that C1's applies_if duplicates the deadline ('within 3 days of confirmation') rather than isolating the trigger condition ('confirmation of a security incident'); this slightly violates the rule to put temporal/triggering conditions in applies_if rather than embedding the deadline there. Otherwise the output is clean and compliant.

Claude Sonnet 4.6

```
```json
[
  {
    "clause_id": "C1",
    "owner": "vendor",
    "action": "notify the Client of any security incident within 3 days
of confirmation",
    "deadline_days": 3,
    "applies_if": "a security incident has been confirmed",
    "excluded_if": "law enforcement prohibits disclosure"
  },
  {
    "clause_id": "C4",
    "owner": "both",
    "action": "maintain commercially reasonable backups",
    "deadline_days": null,
    "applies_if": "during the Term",
    "excluded_if": null
  },
  {
```
```

```
    "clause_id": "C6",
    "owner": "client",
    "action": "provide relevant access credentials",
    "deadline_days": 5,
    "applies_if": "after written onboarding notice has been received",
    "excluded_if": "prohibited by internal policy"
  },
  {
    "clause_id": "C8",
    "owner": "vendor",
    "action": "restore the staging environment within 2 days",
    "deadline_days": 2,
    "applies_if": "during any approved disaster-recovery test",
    "excluded_if": "the Client has not delivered the required seed
data"
  },
  {
    "clause_id": "C10",
    "owner": "both",
    "action": "designate a security contact within 7 days",
    "deadline_days": 7,
    "applies_if": "upon written request by the Client",
    "excluded_if": "a merger-related confidentiality hold is active"
  }
]
```

Strengths: Correctly identified all 5 obligation clauses (C1, C4, C6, C8, C10); Properly excluded definitional (C2), permissive (C3, C7, C9), and consequence-only (C5) clauses; Correct owner normalization including 'both' for C4 and C10; Accurate extraction of deadlines as integers; Exceptions properly placed in excluded_if; Triggering conditions correctly placed in applies_if

Weaknesses: Action fields for C1, C8, and C10 redundantly include the deadline already captured in deadline_days; Minor: action could be more cleanly separated from temporal conditions

The response correctly identifies all five affirmative-obligation clauses and excludes the definitional, permissive, and consequence-only ones per the rules. Owner values are normalized properly, with 'both' correctly applied to C4 ('Each party') and C10 ('each party'). Deadlines are integers where applicable and null for C4. Exceptions (law enforcement, internal policy, seed data, merger hold) are correctly captured in excluded_if. Trigger conditions (confirmation, Term, onboarding notice, DR test, written request) are properly placed in applies_if. The main minor issue is that action strings for C1, C8, and C10 still embed

'within N days' duplicating deadline_days, which slightly violates the spirit of separating temporal conditions from action. Otherwise the JSON is clean and valid.

OVH gpt-oss-120b


```
[
  {
    "clause_id": "C1",
    "owner": "vendor",
    "action": "notify the Client of any security incident",
    "deadline_days": 3,
    "applies_if": "any security incident occurs",
    "excluded_if": "law enforcement prohibits disclosure"
  },
  {
    "clause_id": "C4",
    "owner": "both",
    "action": "maintain commercially reasonable backups",
    "deadline_days": null,
    "applies_if": "during the Term",
    "excluded_if": null
  },
  {
    "clause_id": "C6",
    "owner": "client",
    "action": "provide relevant access credentials",
    "deadline_days": 5,
    "applies_if": "after written onboarding notice",
    "excluded_if": "prohibited by internal policy"
  },
  {
    "clause_id": "C8",
    "owner": "vendor",
    "action": "restore the staging environment",
    "deadline_days": 2,
    "applies_if": "during any approved disaster-recovery test",
    "excluded_if": "Client has not delivered the required seed data"
  },
  {
    "clause_id": "C10",
    "owner": "both",
    "action": "designate a security contact",
    "deadline_days": 7,
    "applies_if": "upon written request by the Client",
    "excluded_if": "while a merger-related confidentiality hold is
active"
```

```
  }
]
```

Strengths: Includes exactly the five affirmative-obligation clauses that should be retained: C1, C4, C6, C8, and C10.; Uses the required JSON array structure and each object contains exactly the required keys.; Correctly normalizes owners to allowed values: vendor, client, both.; Correctly excludes definitional, permissive, and consequence-only clauses such as C2, C3, C5, C7, and C9.; Deadlines are extracted as integers or null, and exceptions are captured in excluded_if.

Weaknesses: C1 applies_if is slightly overinterpreted: 'any security incident occurs' is less faithful than a trigger tied to confirmation, since the source says 'within 3 days of confirmation'.; Some trigger phrasing could be more exact, e.g. C6 might better preserve 'within 5 days after written onboarding notice' as triggered by receipt of written onboarding notice, and C1 should reflect confirmation specifically.

This is a high-quality submission. It correctly includes only C1, C4, C6, C8, and C10, which are the affirmative obligations, and excludes the definitional clause (C2), permissive clauses (C3, C7, C9), and the consequence-only clause (C5). The owner normalization is correct throughout, including mapping 'Each party' to 'both'. Deadline extraction is also correct: 3, 5, 2, and 7 are properly represented as integers, while C4 appropriately uses null. The main issue is slight imprecision in applies_if for C1. The source obligation is to notify within 3 days of confirmation, so the triggering condition is closer to 'upon confirmation of a security incident' than 'any security incident occurs'. That wording shifts the trigger earlier than the clause states. The rest of the applies_if fields are quite good: 'during the Term', 'after written onboarding notice', 'during any approved disaster-recovery test', and 'upon written request by the Client' all sensibly move temporal/triggering language out of the action. Exceptions are also handled correctly in excluded_if. Overall, the candidate follows instructions very well and is nearly fully correct, with only minor fidelity issues in how one trigger condition is phrased.

gpt-5.4

```
[
  {
    "clause_id": "C1",
    "owner": "vendor",
    "action": "notify the Client of any security incident",
    "deadline_days": 3,
    "applies_if": "upon confirmation of a security incident",
    "excluded_if": "law enforcement prohibits disclosure"
  },
  {
```

```
    "clause_id": "C4",
    "owner": "both",
    "action": "maintain commercially reasonable backups",
    "deadline_days": null,
    "applies_if": "during the Term",
    "excluded_if": null
  },
  {
    "clause_id": "C6",
    "owner": "client",
    "action": "provide relevant access credentials",
    "deadline_days": 5,
    "applies_if": "after written onboarding notice",
    "excluded_if": "prohibited by internal policy"
  },
  {
    "clause_id": "C8",
    "owner": "vendor",
    "action": "restore the staging environment",
    "deadline_days": 2,
    "applies_if": "during any approved disaster-recovery test",
    "excluded_if": "the Client has not delivered the required seed
data"
  },
  {
    "clause_id": "C10",
    "owner": "both",
    "action": "designate a security contact",
    "deadline_days": 7,
    "applies_if": "upon written request by the Client",
    "excluded_if": "a merger-related confidentiality hold is active"
  }
]
```


Strengths: Correctly included exactly C1, C4, C6, C8, C10; Proper owner normalization including 'both' for C4 and C10; Clean separation of triggers into applies_if and exceptions into excluded_if; Deadlines extracted as integers or null as required

Weaknesses: C1 applies_if is slightly redundant since the 3-day trigger is tied to confirmation; could be argued as part of deadline semantics; Minor: action strings omit 'shall/must' which is fine but could include more context

The response correctly excludes definitional (C2), permissive (C3, C7, C9), and consequence-only (C5) clauses. All five obligation clauses are present with accurate owner assignment (vendor, both, client, vendor, both). Triggers are properly placed in applies_if (e.g., 'during the Term', 'after written onboarding notice', 'during any approved disaster-recovery test', 'upon written request by the Client'), and exceptions are preserved in

excluded_if. Deadlines are correctly typed as integers or null. JSON structure matches the required schema exactly. Very minor stylistic quibbles only.

gpt-5.4-mini


```
[
  {
    "clause_id": "C1",
    "owner": "vendor",
    "action": "notify the Client of any security incident",
    "deadline_days": 3,
    "applies_if": "after confirmation",
    "excluded_if": "law enforcement prohibits disclosure"
  },
  {
    "clause_id": "C4",
    "owner": "both",
    "action": "maintain commercially reasonable backups",
    "deadline_days": null,
    "applies_if": "during the Term",
    "excluded_if": null
  },
  {
    "clause_id": "C6",
    "owner": "client",
    "action": "provide relevant access credentials",
    "deadline_days": 5,
    "applies_if": "after written onboarding notice",
    "excluded_if": "prohibited by internal policy"
  },
  {
    "clause_id": "C8",
    "owner": "vendor",
    "action": "restore the staging environment",
    "deadline_days": 2,
    "applies_if": "during any approved disaster-recovery test",
    "excluded_if": "the Client has not delivered the required seed
data"
  },
  {
    "clause_id": "C10",
    "owner": "both",
    "action": "designate a security contact",
    "deadline_days": 7,
    "applies_if": "upon written request by the Client",
    "excluded_if": "a merger-related confidentiality hold is active"
```

```
  }
]
```

Strengths: Included exactly the affirmative-obligation clauses requested: C1, C4, C6, C8, and C10.; Used the required JSON array structure with objects containing exactly the specified keys.; Correctly normalized owners to allowed values: vendor, client, both.; Correctly extracted deadline_days as integers or null.; Properly placed triggering/temporal conditions in applies_if and exceptions/blockers in excluded_if.; Correctly excluded definitional, permissive, and consequence-only clauses (C2, C3, C5, C7, C9).

Weaknesses: Minor phrasing simplification in some applies_if fields (e.g., 'after confirmation') could be seen as slightly less faithful than the full original wording, though still correct.; C10 excluded_if omits 'while' and is slightly normalized rather than quoted closely, but meaning is preserved.

The candidate performs extremely well on all core requirements. It returns valid JSON only, includes only the five affirmative-obligation clauses that should be captured (C1, C4, C6, C8, C10), and excludes the clauses that are definitional, permissive, or merely state consequences. Owner normalization is correct throughout: Vendor→vendor, Client→client, Each party→both. Deadline extraction is also correct: 3, null, 5, 2, and 7 respectively. The response properly separates actions from conditions by putting trigger/temporal language into applies_if, such as 'after written onboarding notice' and 'upon written request by the Client', and it preserves exceptions/blockers in excluded_if, including the postposed exception in C8. The wording is concise and semantically accurate. Any issues are extremely minor and limited to slight paraphrasing/normalization of condition language rather than strict textual mirroring. These do not materially affect correctness.

gpt-5.4-nano

```
[
  {
    "clause_id": "C1",
    "owner": "vendor",
    "action": "notify the Client of any security incident",
    "deadline_days": 3,
    "applies_if": "within 3 days of confirmation",
    "excluded_if": "law enforcement prohibits disclosure"
  },
  {
    "clause_id": "C4",
    "owner": "both",
    "action": "maintain commercially reasonable backups",
    "deadline_days": null,
```

```
    "applies_if": "during the Term",
    "excluded_if": null
  },
  {
    "clause_id": "C6",
    "owner": "client",
    "action": "provide relevant access credentials",
    "deadline_days": 5,
    "applies_if": "within 5 days after written onboarding notice",
    "excluded_if": "where prohibited by internal policy"
  },
  {
    "clause_id": "C8",
    "owner": "vendor",
    "action": "restore the staging environment",
    "deadline_days": 2,
    "applies_if": "during any approved disaster-recovery test",
    "excluded_if": "the Client has not delivered the required seed
data"
  },
  {
    "clause_id": "C10",
    "owner": "both",
    "action": "designate a security contact",
    "deadline_days": 7,
    "applies_if": "upon written request by the Client",
    "excluded_if": "while a merger-related confidentiality hold is
active"
  }
]
```


Strengths: Correctly identified all five obligation clauses (C1, C4, C6, C8, C10); Properly excluded definitional (C2), permissive (C3, C7, C9), and consequence-only (C5) clauses; Correct owner normalization including 'both' for C4 and C10; Exceptions properly placed in excluded_if; Deadlines correctly extracted as integers

Weaknesses: C1 applies_if redundantly repeats the deadline ('within 3 days of confirmation') rather than capturing only the trigger ('confirmation of a security incident'); the temporal window is already in deadline_days; C6 applies_if similarly duplicates the deadline instead of isolating the trigger ('written onboarding notice'); Minor: action fields could be slightly more concise but are acceptable

The candidate correctly filters the clause set, keeping exactly the five affirmative obligations and excluding definitional, permissive, and consequence-only clauses. Owner assignments are accurate, including 'both' for bilateral obligations in C4 and C10. Deadlines are correctly parsed as integers, and exceptions/blockers are appropriately captured in excluded_if. The main weakness is that applies_if for C1 and C6 restates the deadline window rather than

isolating the triggering event (e.g., 'confirmation of a security incident' for C1, 'written onboarding notice' for C6). C4, C8, and C10 handle the trigger/condition distinction well. Overall a high-quality, largely compliant response.

extract-incident-records-with-rule-collisions

Structured Output

SYSTEM PROMPT

```
Return valid JSON only. Follow the schema and all transformation rules
exactly. Do not include commentary.
```

USER PROMPT

```
Convert the records below into a JSON array. Each object must contain
exactly these keys:
`timestamp`, `severity`, `service`, `ticket_id`, `ip_addresses`,
`customer_visible`

Rules:
- `timestamp` must be ISO 8601 UTC using `Z`
- `severity` must be one of: `info`, `warning`, `error`, `critical`
- `service` must be lowercase
- `ticket_id` must be a string or null
- `ip_addresses` must be an array
- `customer_visible` must be a boolean
- Omit records with severity `debug`
- Omit records with invalid timestamps
- Mask private IPs in 10.x.x.x, 172.16.x.x to 172.31.x.x, and 192.168.x.x as
`INTERNAL`
- If no ticket is present, use null
- Preserve IP order
- Return JSON only

Records:
1. [2026-04-18 09:14:02 UTC] ERROR Auth ticket=INC-991 visible=yes
src=192.168.1.8 proxy=34.201.7.9 msg="login timeout"
2. [2026-04-18 09:14:05 UTC] DEBUG Billing ticket=INC-992 visible=no
src=10.0.0.4 msg="retry"
3. [2026-04-18 09:15:11 UTC] CRITICAL API visible=yes src=52.48.22.1
mirror=172.20.4.5 msg="upstream failure"
4. [invalid-ts] WARNING Search ticket=INC-993 visible=yes src=18.204.0.2
msg="latency"
5. [2026-04-18 09:16:40 UTC] INFO Worker visible=no msg="job completed"
```

Evaluation criteria: Reward exact omission of debug and invalid timestamp records, correct RFC3339-style UTC timestamps, correct masking of all private ranges including 172.20.x.x, null ticket handling, and strict schema compliance.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**100.00**	#1	3759 ms	908	$0.0309000000	11.01
Claude Sonnet 4.6	**100.00**	#1	3469 ms	703	$0E-10	14.22
OVH gpt-oss-120b	**98.00**	#1	14527 ms	1048	$0.0002675200	9.35
gpt-5.4	**100.00**	#1	3243 ms	624	$0.0042225000	16.03
gpt-5.4-mini	**100.00**	#1	1314 ms	554	$0.0009517500	18.05
gpt-5.4-nano	**98.00**	#1	4493 ms	624	$0.0003484500	15.71

Claude Opus 4.7


```
[
  {
    "timestamp": "2026-04-18T09:14:02Z",
    "severity": "error",
    "service": "auth",
    "ticket_id": "INC-991",
    "ip_addresses": ["INTERNAL", "34.201.7.9"],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:15:11Z",
    "severity": "critical",
    "service": "api",
    "ticket_id": null,
    "ip_addresses": ["52.48.22.1", "INTERNAL"],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:16:40Z",
    "severity": "info",
    "service": "worker",
    "ticket_id": null,
    "ip_addresses": [],
    "customer_visible": false
```

```
  }
]
```

Strengths: Correctly omits the debug record.; Correctly omits the record with an invalid timestamp.; Uses valid ISO 8601 UTC timestamps with trailing Z.; Normalizes severities to allowed lowercase values.; Lowercases service names correctly.; Uses string ticket IDs when present and null when absent.; Masks private IPs correctly, including 192.168.x.x and 172.20.x.x.; Preserves IP address order within each record.; Uses arrays for ip_addresses and booleans for customer_visible.; Returns valid JSON with exactly the required keys in each object.

Weaknesses: No substantive weaknesses.

The candidate response is valid JSON and includes exactly the three records that should remain after applying the omission rules: the DEBUG record is removed and the invalid timestamp record is removed. Each object contains exactly the required keys: timestamp, severity, service, ticket_id, ip_addresses, and customer_visible. Timestamps are correctly converted to ISO 8601 UTC form with a trailing Z. Severity values are normalized to the allowed lowercase set, and service names are lowercase. Ticket handling is correct: INC-991 is preserved as a string, and missing tickets are represented as null. IP masking is correct for private addresses, including 192.168.1.8 and 172.20.4.5, both replaced with INTERNAL, while public IPs are preserved. IP order is also preserved. The final Worker record correctly uses an empty ip_addresses array because no IPs are present. Overall, this is an exact and compliant answer.

Claude Sonnet 4.6

```
```json
[
  {
    "timestamp": "2026-04-18T09:14:02Z",
    "severity": "error",
    "service": "auth",
    "ticket_id": "INC-991",
    "ip_addresses": ["INTERNAL", "34.201.7.9"],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:15:11Z",
    "severity": "critical",
    "service": "api",
    "ticket_id": null,
    "ip_addresses": ["52.48.22.1", "INTERNAL"],
    "customer_visible": true
```
```

```
  },
  {
    "timestamp": "2026-04-18T09:16:40Z",
    "severity": "info",
    "service": "worker",
    "ticket_id": null,
    "ip_addresses": [],
    "customer_visible": false
  }
]
```

Strengths: Correctly omits the debug record.; Correctly omits the record with an invalid timestamp.; Uses valid ISO 8601 UTC timestamps with Z suffix.; Normalizes severity values to allowed lowercase enums.; Converts service names to lowercase.; Uses null when ticket is absent.; Masks private IPs correctly, including 192.168.x.x and 172.20.x.x.; Preserves IP address order.; Provides ip_addresses as arrays and customer_visible as booleans.; Each object contains exactly the required keys and the output is valid JSON only.

The candidate output exactly matches the requested JSON-array transformation. It includes only the valid non-debug records, formats timestamps correctly as RFC3339/ISO 8601 UTC with a trailing Z, lowercases both severity and service, uses null for missing tickets, preserves IP order, masks private addresses as INTERNAL, and represents customer_visible as booleans. The worker record correctly has an empty ip_addresses array because no IPs are present. Schema compliance is strict: every object has exactly the required six keys and no extras.

OVH gpt-oss-120b

```
[
  {
    "timestamp": "2026-04-18T09:14:02Z",
    "severity": "error",
    "service": "auth",
    "ticket_id": "INC-991",
    "ip_addresses": [
      "INTERNAL",
      "34.201.7.9"
    ],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:15:11Z",
    "severity": "critical",
```

```
    "service": "api",
    "ticket_id": null,
    "ip_addresses": [
      "52.48.22.1",
      "INTERNAL"
    ],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:16:40Z",
    "severity": "info",
    "service": "worker",
    "ticket_id": null,
    "ip_addresses": [],
    "customer_visible": false
  }
]
```


Strengths: Correctly omitted debug and invalid timestamp records; Properly masked 192.168.x.x and 172.20.x.x as INTERNAL; Correct ISO 8601 UTC timestamps with Z; Preserved IP order; Correct null ticket handling; Lowercase service names; Proper boolean conversion of visible=yes/no

Weaknesses: None significant

The response correctly processes all 5 records: omits record 2 (debug) and record 4 (invalid timestamp), masks both private IPs (192.168.1.8 and 172.20.4.5) as INTERNAL while preserving order, converts timestamps to ISO 8601 Z format, lowercases services, handles null ticket_id for records without tickets, and correctly converts yes/no to booleans. Schema is strict and complete.

gpt-5.4


```
[
  {
    "timestamp": "2026-04-18T09:14:02Z",
    "severity": "error",
    "service": "auth",
    "ticket_id": "INC-991",
    "ip_addresses": [
      "INTERNAL",
      "34.201.7.9"
    ],
    "customer_visible": true
  },
```

```
  {
    "timestamp": "2026-04-18T09:15:11Z",
    "severity": "critical",
    "service": "api",
    "ticket_id": null,
    "ip_addresses": [
      "52.48.22.1",
      "INTERNAL"
    ],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:16:40Z",
    "severity": "info",
    "service": "worker",
    "ticket_id": null,
    "ip_addresses": [],
    "customer_visible": false
  }
]
```

Strengths: Correctly omits the debug record.; Correctly omits the record with an invalid timestamp.; Uses valid ISO 8601 UTC timestamps with Z suffix.; Normalizes severity values to the allowed lowercase set.; Lowercases service names correctly.; Uses null for missing ticket IDs.; Masks private IPs correctly, including 192.168.x.x and 172.20.x.x.; Preserves IP address order within each record.; Returns valid JSON only with exactly the required keys per object.; Correctly sets customer_visible as booleans.

The candidate fully satisfies the prompt. It includes only the valid non-debug records (1, 3, and 5), converts timestamps to ISO 8601 UTC format with a trailing Z, lowercases service names, maps severities to the required lowercase values, uses string ticket IDs or null appropriately, preserves IP order, masks private addresses as INTERNAL, and outputs booleans for customer_visible. Each object contains exactly the required keys and the response is valid JSON only.

gpt-5.4-mini


```
[{"timestamp":"2026-04-18T09:14:02Z","severity":"error","service":"auth","ticket_id":"INC-991","ip_addresses":["INTERNAL","34.201.7.9"],"customer_visible":true},{"timestamp":"2026-04-18T09:15:11Z","severity":"critical","service":"api","ticket_id":null,"ip_addresses":["52.48.22.1","INTERNAL"],"customer_visible":true},{"timestamp":"2026-04-
```

18T09:16:40Z","severity":"info","service":"worker","ticket_id":null,"ip_addresses":[],"customer_visible":false}]

Strengths: Correctly omits the debug record.; Correctly omits the record with invalid timestamp.; Uses valid ISO 8601 UTC timestamps with Z.; Normalizes severity values to allowed lowercase forms.; Normalizes service names to lowercase.; Uses null for missing ticket_id values.; Masks private IPs correctly, including 192.168.x.x and 172.20.x.x.; Preserves IP address order.; Uses arrays for ip_addresses and booleans for customer_visible.; Returns JSON only with the required schema.

Weaknesses: No meaningful weaknesses; output appears fully compliant.

The candidate precisely follows the transformation rules. Record 1 is converted correctly with timestamp 2026-04-18T09:14:02Z, severity error, lowercase service auth, ticket_id as a string, private 192.168.1.8 masked to INTERNAL, public proxy preserved, and customer_visible set to true. Record 2 is correctly omitted because severity is DEBUG. Record 3 is correctly included with timestamp 2026-04-18T09:15:11Z, severity critical, lowercase service api, missing ticket represented as null, public IP preserved first, and private 172.20.4.5 correctly masked to INTERNAL. Record 4 is correctly omitted due to invalid timestamp. Record 5 is correctly included with timestamp 2026-04-18T09:16:40Z, severity info, lowercase service worker, null ticket_id, empty ip_addresses array, and customer_visible false. The response is valid JSON and each object contains exactly the required keys.

gpt-5.4-nano


```
[
  {
    "timestamp": "2026-04-18T09:14:02Z",
    "severity": "error",
    "service": "auth",
    "ticket_id": "INC-991",
    "ip_addresses": [
      "INTERNAL",
      "34.201.7.9"
    ],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:15:11Z",
    "severity": "critical",
    "service": "api",
    "ticket_id": null,
    "ip_addresses": [
      "52.48.22.1",
```

```
      "INTERNAL"
    ],
    "customer_visible": true
  },
  {
    "timestamp": "2026-04-18T09:16:40Z",
    "severity": "info",
    "service": "worker",
    "ticket_id": null,
    "ip_addresses": [],
    "customer_visible": false
  }
]
```


Strengths: Correctly omitted debug and invalid-timestamp records; Properly masked 192.168.x.x and 172.20.x.x as INTERNAL; ISO 8601 UTC with Z format; Correct null ticket handling and preserved IP order; Strict schema compliance

Weaknesses: Minor: none of substance

All five rules applied correctly: record 2 (debug) and record 4 (invalid-ts) omitted; timestamps converted to RFC3339 Z form; services lowercased; severities normalized; private IPs (192.168.1.8 and 172.20.4.5) masked as INTERNAL while public IPs preserved in order; tickets set to null where absent; customer_visible converted to boolean. Schema keys match exactly.

summarize-policy-with-exception-hierarchy

Summarization

SYSTEM PROMPT

```
You are a compliance-focused summarizer. Preserve the exact scope of every
exception and counter-exception. Do not merge distinct consequence tracks.
Do not infer conditions that are not explicitly stated.
```

USER PROMPT

```
Summarize the policy below in exactly 7 bullets using these labels in this
exact order:
`Scope`
`Default prohibitions`
`Role exception`
`Counter-exception`
`Mandatory disclosure`
`Consequence track A`
`Consequence track B`

Rules for your summary:
- Each bullet must begin with its label followed by a colon
- Do not merge `Consequence track A` and `Consequence track B` into one
bullet
- The `Role exception` bullet must name the eligible role and both
conditions that must be simultaneously true
- The `Counter-exception` bullet must state who it applies to and the single
condition that re-activates the prohibition
- Do not quote the source text. Do not add examples. Do not exceed one
sentence per bullet.

Policy:
This policy governs all personnel with elevated system privileges, including
full-time engineers, on-call contractors, and third-party auditors operating
under a signed data-processing agreement. By default, no covered person may
access production databases directly, export raw customer records, or modify
infrastructure configuration outside a change-management ticket. Senior
engineers who hold an active security clearance and whose team lead has
filed a standing access request for the current quarter may perform direct
production reads for incident diagnosis. However, this exception does not
apply to senior engineers who are currently under a disciplinary review,
even if all other conditions are met. Any access that results in the export
of more than 500 customer records — whether intentional or incidental — must
be disclosed to the Data Protection Officer within 4 business hours and
logged in the incident register. Personnel who violate the default
prohibitions through negligence face mandatory retraining and a 30-day
```

```
privilege suspension; a second negligent violation within 12 months triggers
automatic escalation to HR. Personnel who commit intentional or concealed
violations face immediate privilege revocation, contract termination, and
referral to relevant regulatory authorities without prior warning.
```

Evaluation criteria: Reward: correct 7-bullet structure in exact label order; Scope covering all three personnel categories; Default prohibitions listing all three; Role exception naming senior engineers with both conditions (clearance + standing request); Counter-exception scoped to disciplinary review only; Mandatory disclosure with 500-record threshold and 4-hour window; Consequence track A for negligence (retraining + suspension + 12-month escalation trigger); Consequence track B for intentional/concealed (revocation + termination + regulatory referral). Penalize: merging consequence tracks, omitting the disciplinary-review counter-exception, dropping the 12-month recidivism condition, or exceeding one sentence per bullet.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Opus 4.7	**99.00**	#1	4448 ms	1121	$0.0422550000	8.83
Claude Sonnet 4.6	**95.00**	#1	4187 ms	756	$0E-10	12.57
OVH gpt-oss-120b	**99.00**	#1	7665 ms	1251	$0.0003445600	7.91
gpt-5.4	**99.00**	#1	3695 ms	667	$0.0047050000	14.84
gpt-5.4-mini	**100.00**	#1	3163 ms	684	$0.0014880000	14.62
gpt-5.4-nano	**99.00**	#1	2785 ms	662	$0.0003823000	14.95

Claude Opus 4.7

```
Scope: This policy covers all personnel with elevated system
privileges, specifically full-time engineers, on-call contractors, and
third-party auditors operating under a signed data-processing
agreement.
Default prohibitions: Covered persons may not directly access
production databases, export raw customer records, or modify
infrastructure configuration outside a change-management ticket.
Role exception: Senior engineers may perform direct production reads
for incident diagnosis when they hold an active security clearance and
```

```
their team lead has filed a standing access request for the current
quarter.
Counter-exception: The exception does not apply to senior engineers who
are currently under a disciplinary review, which re-activates the
default prohibition regardless of other conditions.
Mandatory disclosure: Any access resulting in the export of more than
500 customer records, whether intentional or incidental, must be
disclosed to the Data Protection Officer within 4 business hours and
logged in the incident register.
Consequence track A: Negligent violations of the default prohibitions
result in mandatory retraining and a 30-day privilege suspension, with
a second negligent violation within 12 months triggering automatic
escalation to HR.
Consequence track B: Intentional or concealed violations result in
immediate privilege revocation, contract termination, and referral to
relevant regulatory authorities without prior warning.
```

Strengths: Uses exactly 7 bullets with the required labels in the exact order.; Accurately covers scope, including all three personnel categories and the signed data-processing agreement qualifier for third-party auditors.; Lists all three default prohibitions without omission.; Correctly identifies the eligible role as senior engineers and includes both required simultaneous conditions: active security clearance and a standing access request for the current quarter filed by the team lead.; Correctly states the counter-exception applies to senior engineers under disciplinary review and that this re-activates the prohibition.; Includes the mandatory disclosure threshold of more than 500 customer records, the 4 business hour deadline, and the logging requirement.; Keeps consequence tracks separate and accurately states the negligence path, including retraining, 30-day suspension, and second negligent violation within 12 months escalating to HR, as well as the intentional/concealed path of revocation, termination, and regulatory referral.

Weaknesses: Formatting is presented as labeled lines rather than markdown bullets, though the required label structure is fully preserved.

The candidate response closely matches the source policy and follows the requested structure with seven labeled items in the exact required order. It includes all required scope categories, all three default prohibitions, the role exception with both necessary conditions, the disciplinary-review counter-exception, the disclosure threshold and timing, and both consequence tracks separately with the 12-month recidivism trigger preserved. Each item is limited to one sentence and avoids quoting the source text or adding examples. The only slight issue is that the output is formatted as labeled lines rather than explicit bullets, but functionally it satisfies the structural intent extremely well.

Claude Sonnet 4.6

```
- **Scope:** This policy applies to all personnel with elevated system
privileges, including full-time engineers, on-call contractors, and
third-party auditors operating under a signed data-processing
agreement.
- **Default prohibitions:** No covered person may directly access
production databases, export raw customer records, or modify
infrastructure configuration outside a change-management ticket.
- **Role exception:** Senior engineers may perform direct production
reads for incident diagnosis only when they simultaneously hold an
active security clearance and have had their team lead file a standing
access request for the current quarter.
- **Counter-exception:** The role exception does not apply to senior
engineers who are currently under a disciplinary review, even if all
other conditions are otherwise satisfied.
- **Mandatory disclosure:** Any access resulting in the export of more
than 500 customer records, whether intentional or incidental, must be
reported to the Data Protection Officer within 4 business hours and
logged in the incident register.
- **Consequence track A:** Personnel who violate default prohibitions
through negligence face mandatory retraining and a 30-day privilege
suspension, with a second negligent violation within 12 months
triggering automatic escalation to HR.
- **Consequence track B:** Personnel who commit intentional or
concealed violations face immediate privilege revocation, contract
termination, and referral to relevant regulatory authorities without
prior warning.
```

Strengths: All 7 labels present in exact order with colons; Scope covers all three personnel categories; Role exception names senior engineers with both simultaneous conditions; Counter-exception correctly scoped to disciplinary review; Mandatory disclosure includes 500-record threshold and 4-hour window; Consequence tracks kept separate with all required elements including 12-month recidivism

Weaknesses: Minor near-quoting of source phrases (e.g., 'signed data-processing agreement', 'without prior warning') which the instructions arguably discouraged; Bold markdown on labels slightly deviates from the plain 'label:' format shown

The response correctly produces exactly seven bullets in the required label order, each as a single sentence. Scope enumerates all three personnel types; default prohibitions lists all three prohibited actions; the role exception identifies senior engineers and ties both the active security clearance and the standing quarterly access request as simultaneous requirements; the counter-exception cleanly isolates disciplinary review as the reactivating condition. The disclosure bullet preserves both the >500 record threshold and the 4 business hour window plus the incident register. Consequence tracks A and B are kept distinct, with the 12-month recidivism escalation preserved in A and revocation/termination/regulatory referral in B. Small issues: several phrases closely mirror

the source wording despite the no-quoting instruction, and the use of bold markdown on labels is a minor stylistic deviation, but these do not materially harm fidelity.

OVH gpt-oss-120b

```
- Scope: Applies to all personnel with elevated system privileges,
including full-time engineers, on-call contractors, and third-party
auditors under a signed data-processing agreement.
- Default prohibitions: No covered person may directly access
production databases, export raw customer records, or modify
infrastructure configuration without a change-management ticket.
- Role exception: Senior engineers who hold an active security
clearance **and** whose team lead has filed a standing access request
for the current quarter may perform direct production reads for
incident diagnosis.
- Counter-exception: The exception is nullified for senior engineers
who are currently under disciplinary review.
- Mandatory disclosure: Any export of more than 500 customer records,
whether intentional or incidental, must be disclosed to the Data
Protection Officer within 4 business hours and logged in the incident
register.
- Consequence track A: Negligent violations result in mandatory
retraining, a 30-day privilege suspension, and a second negligent
violation within 12 months escalates to HR.
- Consequence track B: Intentional or concealed violations cause
immediate privilege revocation, contract termination, and referral to
relevant regulatory authorities without prior warning.
```

Strengths: Uses exactly 7 bullets with the required labels in the exact specified order.; Covers all scope categories: full-time engineers, on-call contractors, and third-party auditors under a signed data-processing agreement.; Includes all three default prohibitions.; Correctly identifies the eligible role as senior engineers and states both required conditions for the exception.; Correctly states the counter-exception for senior engineers under disciplinary review.; Includes the mandatory disclosure threshold, time window, and logging requirement.; Keeps consequence track A and B separate and includes the 12-month repeat-negligence escalation condition.

Weaknesses: The counter-exception bullet says the exception is nullified rather than explicitly saying the prohibition is re-activated, though the meaning is effectively correct.; Minor formatting emphasis in the role exception is unnecessary but not noncompliant.

Candidate A is highly accurate and nearly ideal. It follows the exact 7-bullet structure, uses the required labels in the correct order, and keeps each bullet to one sentence. The Scope

bullet includes all three covered personnel categories and the signed data-processing agreement condition for third-party auditors. The Default prohibitions bullet lists all three prohibited actions. The Role exception correctly names senior engineers and includes both simultaneous conditions: active security clearance and a standing access request filed by the team lead for the current quarter, along with the permitted activity and purpose. The Counter-exception correctly applies to senior engineers under disciplinary review; while it could more literally say the prohibition is re-activated, its meaning is clear and faithful. The Mandatory disclosure bullet accurately includes exports of more than 500 customer records, whether intentional or incidental, the 4-business-hour disclosure deadline, and the logging requirement. Consequence track A correctly covers negligence, mandatory retraining, 30-day privilege suspension, and the second negligent violation within 12 months escalating to HR. Consequence track B correctly covers intentional or concealed violations, immediate privilege revocation, contract termination, and regulatory referral without prior warning. Overall, this is an excellent summary with only negligible wording imperfections.

gpt-5.4

```
- Scope: This policy applies to all personnel with elevated system
privileges, including full-time engineers, on-call contractors, and
third-party auditors operating under a signed data-processing
agreement.
- Default prohibitions: Covered personnel may not directly access
production databases, export raw customer records, or modify
infrastructure configuration outside a change-management ticket.
- Role exception: Senior engineers may perform direct production reads
for incident diagnosis only if they have an active security clearance
and their team lead has filed a standing access request for the current
quarter.
- Counter-exception: The role exception does not apply to senior
engineers who are currently under a disciplinary review.
- Mandatory disclosure: Any access resulting in the export of more than
500 customer records, whether intentional or incidental, must be
disclosed to the Data Protection Officer within 4 business hours and
logged in the incident register.
- Consequence track A: Negligent violations of the default prohibitions
result in mandatory retraining and a 30-day privilege suspension, and a
second negligent violation within 12 months triggers automatic
escalation to HR.
- Consequence track B: Intentional or concealed violations result in
immediate privilege revocation, contract termination, and referral to
relevant regulatory authorities without prior warning.
```

Strengths: Uses exactly 7 bullets with the required labels in the exact specified order.; Covers all three scope categories: full-time engineers, on-call contractors, and third-

party auditors under a signed data-processing agreement.; Lists all three default prohibitions.; Correctly identifies the eligible role as senior engineers and includes both required simultaneous conditions: active security clearance and a standing access request for the current quarter filed by the team lead.; States the counter-exception correctly for senior engineers under disciplinary review.; Includes the mandatory disclosure trigger of more than 500 customer records, the 4-business-hour deadline, and the incident-register logging requirement.; Separates consequence track A and B correctly, including the 12-month repeat-negligence escalation to HR and the intentional/concealed violation penalties.

Weaknesses: The counter-exception bullet could be even more explicit that disciplinary review is the single condition that re-activates the prohibition, though it is still clear and correct.; Minor wording remains close to the source text, but it does not quote it and stays within the rules.

Candidate A is an exceptionally strong answer. It follows the required format exactly: 7 bullets, each beginning with the mandated label and presented in the exact order specified. The Scope bullet correctly includes all covered personnel categories. The Default prohibitions bullet includes all three prohibited actions. The Role exception bullet correctly names senior engineers and states both required conditions that must simultaneously be true: active security clearance and a standing access request for the current quarter filed by the team lead. The Counter-exception bullet correctly limits the exception for senior engineers currently under disciplinary review; this captures the intended reactivation of the prohibition, though it could have been phrased even more explicitly in those terms. The Mandatory disclosure bullet accurately includes the threshold of more than 500 customer records, applies whether intentional or incidental, and includes both required actions: disclosure to the Data Protection Officer within 4 business hours and logging in the incident register. Consequence track A accurately captures negligent violations, mandatory retraining, 30-day privilege suspension, and the second negligent violation within 12 months leading to automatic HR escalation. Consequence track B correctly states immediate privilege revocation, contract termination, and referral to relevant regulatory authorities without prior warning for intentional or concealed violations. Each bullet is one sentence, no examples are added, and there are no substantive omissions.

gpt-5.4-mini

```
Scope: This policy applies to all personnel with elevated system
privileges, including full-time engineers, on-call contractors, and
third-party auditors operating under a signed data-processing
agreement.

Default prohibitions: Covered persons may not access production
databases directly, export raw customer records, or modify
```

```
infrastructure configuration outside a change-management ticket.

Role exception: Senior engineers may perform direct production reads
for incident diagnosis only if they have an active security clearance
and their team lead has filed a standing access request for the current
quarter.

Counter-exception: This exception does not apply to senior engineers
who are currently under a disciplinary review, and the prohibition
reactivates if that condition is present even when the other conditions
are met.

Mandatory disclosure: Any access that results in the export of more
than 500 customer records, whether intentional or incidental, must be
disclosed to the Data Protection Officer within 4 business hours and
logged in the incident register.

Consequence track A: Negligent violations of the default prohibitions
require mandatory retraining and a 30-day privilege suspension, and a
second negligent violation within 12 months triggers automatic
escalation to HR.

Consequence track B: Intentional or concealed violations result in
immediate privilege revocation, contract termination, and referral to
relevant regulatory authorities without prior warning.
```

Strengths: Uses exactly 7 bullets with the required labels in the exact specified order.; Accurately covers scope, including all three personnel categories.; Lists all three default prohibitions without omission.; Correctly names senior engineers and includes both required simultaneous conditions for the role exception.; Correctly states the counter-exception applies to senior engineers under disciplinary review and that this condition re-activates the prohibition.; Includes the mandatory disclosure threshold, intentional-or-incidental applicability, 4-business-hour deadline, and incident register logging requirement.; Separates the two consequence tracks and correctly includes the 12-month repeat-negligence escalation trigger and the intentional/concealed sanctions.

Weaknesses: The counter-exception bullet is slightly more verbose than necessary, though still compliant and clear.

The candidate response is exceptionally strong and appears fully compliant with the benchmark prompt. It provides exactly seven bullets, each beginning with the required label followed by a colon, and preserves the exact label order. The Scope bullet correctly includes all covered personnel categories: full-time engineers, on-call contractors, and third-party auditors under a signed data-processing agreement. The Default prohibitions bullet includes all three prohibited actions: direct production database access, raw customer record export, and infrastructure configuration changes outside a change-management ticket. The Role

exception bullet correctly identifies senior engineers as the eligible role and includes both necessary conditions that must simultaneously be true: active security clearance and a standing access request filed by the team lead for the current quarter. The Counter-exception bullet properly limits applicability to senior engineers under disciplinary review and clearly states that this condition causes the prohibition to apply even if the other exception conditions are met. The Mandatory disclosure bullet accurately captures the more-than-500-record threshold, the fact that both intentional and incidental exports are covered, the 4-business-hour reporting deadline to the Data Protection Officer, and the logging requirement. Consequence track A correctly addresses negligent violations with mandatory retraining, a 30-day privilege suspension, and automatic HR escalation for a second negligent violation within 12 months. Consequence track B correctly addresses intentional or concealed violations with immediate privilege revocation, contract termination, and referral to relevant regulatory authorities without prior warning. Each bullet is one sentence, no examples are added, and the source is not quoted verbatim in a problematic way. This is a near-perfect benchmark answer.

gpt-5.4-nano

```
- Scope: Covers all personnel with elevated system privileges,
including full-time engineers, on-call contractors, and third-party
auditors operating under a signed data-processing agreement.
- Default prohibitions: Covered persons may not directly access
production databases, export raw customer records, or modify
infrastructure configuration outside a change-management ticket.
- Role exception: Senior engineers may directly read production for
incident diagnosis only if they hold an active security clearance and
their team lead has filed a standing access request for the current
quarter.
- Counter-exception: The prohibition is re-activated for senior
engineers who are currently under disciplinary review, regardless of
meeting the other conditions.
- Mandatory disclosure: Any access resulting in the export of more than
500 customer records must be disclosed to the Data Protection Officer
within 4 business hours and logged in the incident register.
- Consequence track A: Negligent violations face mandatory retraining
and a 30-day privilege suspension, and a second negligent violation
within 12 months triggers automatic escalation to HR.
- Consequence track B: Intentional or concealed violations face
immediate privilege revocation, contract termination, and referral to
relevant regulatory authorities without prior warning.
```

Strengths: Uses exactly 7 bullets with the required labels in the exact specified order.; Covers all three scope categories: full-time engineers, on-call contractors, and third-party auditors under a signed data-processing agreement.; Lists all three default

prohibitions clearly.; Correctly identifies the eligible role as senior engineers and includes both required simultaneous conditions: active security clearance and a current-quarter standing access request filed by the team lead.; States the counter-exception correctly for senior engineers under disciplinary review.; Includes the mandatory disclosure threshold, recipient, deadline, and incident-register logging requirement.; Keeps consequence track A and B separate and accurately captures the 12-month repeat-negligence escalation condition and the intentional/concealed violation penalties.; Maintains one sentence per bullet and avoids quoting the source text.

Weaknesses: The mandatory disclosure bullet omits the nuance that the export threshold applies whether the export was intentional or incidental.

This is a near-perfect summary. It uses exactly seven bullets, each begins with the required label, and the labels appear in the exact mandated order. The scope bullet includes all covered personnel categories. The default prohibitions bullet captures all three prohibited actions. The role exception correctly names senior engineers and includes both conditions that must be true at the same time: active security clearance and a current-quarter standing access request filed by the team lead. The counter-exception properly applies to senior engineers under disciplinary review and clearly re-activates the prohibition. The mandatory disclosure bullet includes the more-than-500-record threshold, the requirement to notify the Data Protection Officer within 4 business hours, and the requirement to log the event in the incident register. The two consequence tracks are separated correctly, with track A including retraining, 30-day suspension, and the second-negligent-violation-within-12-months escalation to HR, and track B including immediate privilege revocation, contract termination, and regulatory referral without prior warning. The only minor omission is that the mandatory disclosure rule in the source applies whether the export was intentional or incidental, and that nuance is not explicitly preserved here.